Patrones de diseño en el código generado con IA

Informe de experimento sobre el sistema ProFit Studio (apps plans y health)

Actividad: Patrones de diseño en el código generado con IA

Equipo: [COMPLETAR: nombres de las personas integrantes]

Curso y docente: [COMPLETAR]

Fecha: 15 de octubre de 2026

Repositorio: https://github.com/Jdarenas22/profit-studio

Tabla de contenido

Resumen

1. Prompt de generación
    1.1 Herramienta y modelos utilizados
    1.2 El texto del prompt de generación
    1.3 ¿El prompt pedía patrones, principios o una arquitectura?
    1.4 Prompt usado en el experimento de reformulación
2. Consulta al asistente
    2.1 Diseño de la consulta
    2.2 Verificación de la respuesta
3. Inventario de patrones aplicados
4. Inventario de patrones omitidos
5. Clasificación: indicado o espontáneo
    5.1 Qué tipo de patrones aplica la IA por sí sola y cuáles omite
6. Experimento de reformulación
    6.1 Pregunta y diseño
    6.2 Resultados
    6.3 Comparación argumentada
7. Conclusiones y recomendación
    7.1 Conclusiones
    7.2 Qué indicar en un prompt de generación, cuándo y cómo
    7.3 Versión mejorada del prompt de generación
    7.4 Limitaciones del experimento
    7.5 Hallazgo práctico fuera del alcance de la actividad
8. Registro de uso de IA
9. Material de consulta

Anexo A. Prompt de consulta (íntegro)

Anexo B. Respuesta íntegra del asistente

Anexo C. Prompts del experimento
    C.1 Prompt A (control), íntegro
    C.2 Prompt B (reformulado): única diferencia con A
    C.3 Prompt de ampliación (estado «vencido»), versión para A

Anexo D. Fragmentos de código comparados
    D.1 Original (V0): apps/plans/services.py, líneas 482 a 503

D.2 Control A: constantes y archive_plan (plan_lifecycle.py, líneas 43-47 y 144-161)

D.3 B con State: estados y contexto (plan_lifecycle.py, líneas 90-127, 186-194 y 262-274)

D.4 Diff de la ampliación en A (plan_lifecycle.py)

D.5 Diff de la ampliación en B (plan_lifecycle.py)

Anexo E. Registro de verificaciones

Resumen

Este informe documenta un experimento sobre el código que Claude Code generó para ProFit Studio, una aplicación Django de gestión de un gimnasio. Un asistente de IA, consultado en una conversación nueva y sin el historial de la generación, identificó 14 patrones GoF y 9 patrones GRASP en las apps `plans` y `health`, y señaló 15 patrones que pudieron aplicarse y no se aplicaron. Se verificó contra el código la mayoría de las cifras que sustentan los hallazgos. Después se regeneró un módulo acotado (el ciclo de vida del plan) en tres versiones y se midió cuánto cuesta ampliarlo.

Hallazgos principales: (1) la IA aplicó por sí sola patrones ligeros y funcionales y asignó responsabilidades con criterio GRASP, y siguió con fidelidad las interfaces que el contrato de diseño pedía; (2) omitió unificar código duplicado que ya empezó a divergir, y no aplicó State, Null Object ni Specification; (3) al pedirle State de forma explícita lo incorporó, pero con más código y mayor costo de ampliación y sin un beneficio claro para este módulo; (4) por eso se recomienda pedir principios con su límite y una auditoría final, no nombres de patrones.

Nota sobre la elaboración. El experimento y este informe los ejecutó Claude Code (modelo Claude Sonnet 5.5) a pedido del equipo. Lo marcado como [COMPLETAR] son datos que solo el equipo conoce, y las valoraciones de relevancia de la Sección 4 son propuestas que el equipo debe revisar y confirmar. El registro completo del uso de IA está en la Sección 8.

1. Prompt de generación

1.1 Herramienta y modelos utilizados

El código se generó con Claude Code (interfaz de línea de comandos de Anthropic) en sesiones sucesivas entre el 6 de mayo y el 15 de octubre de 2026. El repositorio tiene 78 commits; 55 llevan una firma de coautoría que identifica el modelo (Tabla 1). Los otros 23 no indican modelo.

Tabla 1. Modelos que firman los commits del repositorio (a 15 de octubre de 2026)

Modelo	Commits firmados	Relación con las apps analizadas
Claude Sonnet 4.6	23	Ninguno toca `apps/plans` ni `apps/health`
Claude Sonnet 5.5	31	11 tocan `apps/plans` y 2 tocan `apps/health`

Modelo	Commits firmados	Relación con las apps analizadas
Claude Opus 5.5	1	Fase B (commit `42ed7e0`): ejercicio con IA, motor de reglas y cliente de IA

Fuente: `git log` sobre `main`. El tamaño del código de producción de las dos apps analizadas es de 8 245 líneas (6 344 en `plans` y 1 901 en `health`, sin pruebas ni migraciones).

1.2 El texto del prompt de generación

El texto íntegro de los prompts con los que se generó el código no se conserva en el repositorio: se dieron en conversaciones de Claude Code que no se guardaron como documentos. Lo que sí quedó escrito y guió la generación son tres fuentes (Tabla 2).

Tabla 2. Fuentes escritas que guiaron la generación

Fuente	Qué pide (citas textuales)
Instrucciones del agente `arquitecto-backend` (archivo de configuración de Claude Code del equipo)	«Todo backend que diseñes, construyas o revises sigue estrictamente Clean Architecture, OWASP y The Twelve-Factor App.» «Separa claramente las rutas/controladores (gestión HTTP), los servicios (lógica de negocio abstracta) y los repositorios (acceso a datos).»
Contrato `docs/contracts/plan-ia.md` (escrito por el agente y aprobado por la persona antes de construir)	«Vistas (HTTP) -> `services.py` (casos de uso) -> modelos/consultas. Reglas puras en `plans/rules/` (sin Django ni red). IA detrás de una interfaz en `plans/ai/`» (§1). Interfaz `AIClient.generate_json(…)` «se inyecta con `get_ai_client()`» (§4.3). Cola en base de datos con `transaction.on_commit` (§6.1). Estados del plan descritos en prosa (§8.1).
`CLAUDE.md` del proyecto	Estructura de apps, modelo de roles, convenciones de seguridad y de pruebas.

[COMPLETAR por el equipo: pegar aquí el texto íntegro de los mensajes con los que se pidió cada fase, si se conservan en el historial de Claude Code.]

1.3 ¿El prompt pedía patrones, principios o una arquitectura?

Pedía **arquitectura y principios generales**: Clean Architecture, separación entre controladores, servicios y repositorios, Twelve-Factor y OWASP. **No pedía ningún patrón de diseño por su nombre.** Se comprobó con `git grep` sobre todos los archivos versionados: ninguno menciona *strategy*, *factory method*, *abstract factory*, *singleton*, *observer pattern*, *template method*, *state pattern*, *repository pattern*, *clean architecture*, *SOLID*, *GRASP* ni «design pattern» (la búsqueda no distingue mayúsculas). Esa ausencia es el punto de partida del experimento: cualquier patrón GoF o GRASP presente en el código lo eligió el asistente, salvo los que el contrato describe como decisión estructural (Sección 5).

1.4 Prompt usado en el experimento de reformulación

Como el prompt original no existe, el experimento de la Sección 6 parte de una **reconstrucción**: la especificación funcional del ciclo de vida del plan (tomada del comportamiento del código y de la sección 8 del contrato) más las pautas generales de arquitectura del agente. No nombra patrones. Está íntegro en el Anexo C.

2. Consulta al asistente

2.1 Diseño de la consulta

Siguiendo el paso 2 de la actividad, la consulta se hizo en una **conversación nueva**, sin el historial de la generación: un agente de Claude Code que arranca sin contexto previo (modelo `claude-sonnet-5-5`, según el propio agente). Se le pidió analizar solo el código de producción de `apps/plans` y `apps/health` (46 archivos), en modo de solo lectura, sin leer `CLAUDE.md`, la carpeta `docs/` ni el historial de git, y sin forzar patrones: solo listar uno si podía señalar su lugar concreto. Las preguntas fueron dos: qué patrones GoF y GRASP contiene el código (con descripción, ubicación, evidencia y grado de certeza) y cuáles pudo o debió aplicar y no aplicó (con el problema que resolvería, el beneficio y el costo). El prompt exacto está en el Anexo A y la respuesta íntegra en el Anexo B.

2.2 Verificación de la respuesta

Una IA puede inventar ubicaciones o cifras, así que antes de usar la respuesta el equipo contrastó con el código las afirmaciones que sustentan los hallazgos (Tabla 3). Las demás se toman de la consulta y se marcan «No» en la columna *Verificado* de las tablas 4 a 6.

Tabla 3. Afirmaciones de la consulta contrastadas con el código

Afirmación del asistente	Resultado de la verificación
Tamaño de `services.py` 559 líneas, `nutrition.py` 520, `jobs.py` 479, `views.py` 670	Coincide exactamente
10 creaciones manuales de `PlanEvent` (6 en `services.py` y 4 en `nutrition.py`)	Coincide exactamente
10 comprobaciones de «adaptación nula» en `jobs.py`	Coincide (10)
13 consultas `PlanGenerationJob.objects.filter(pk=…)` en `jobs.py`	Coincide exactamente
El bloque de cierre con `lost_lock` está repetido en los tres ejecutores	Coincide: `jobs.py` líneas 330, 434 y 467

Afirmación del asistente	Resultado de la verificación
`jobs.py` usa `client.is_simulated` y `client.model_name` sin preguntar el tipo de cliente	Coincide: líneas 332-333 y 436-438
Tabla de despacho `_RUNNERS` en `jobs.py`	Coincide: línea 475
Imports locales con el comentario «evita el import circular»	Coincide parcialmente: dos de las tres ubicaciones citadas (`services.py` líneas 288 y 506)
`request_plan` evita repetir el aviso de bloqueo en el día y `request_alternative` no lo hace (divergencia)	Coincide: `services.py` líneas 303-310 y la rama de bloqueo de `nutrition.request_alternative`
La regla «¿se puede aprobar?» se evalúa en la vista y en el servicio	Coincide: `views.py` líneas 301-303 y `services.approve_plan`
«Unas 30 comparaciones por tipo de plan»	**No reproducible**: 56 líneas mencionan las constantes de tipo (incluidos imports y definiciones). La cifra es imprecisa; la tendencia (comparaciones repartidas en cinco módulos) es cierta

3. Inventario de patrones aplicados

Las tablas 4 y 5 reúnen los patrones que el asistente informa. Los patrones GoF son los del catálogo de Gamma et al. (1994) y los GRASP son los nueve de Larman (2004). El asistente también descartó, con su razón, once patrones GoF «buscados y no encontrados» (Singleton, Abstract Factory, Builder, Prototype, Composite, Bridge, Proxy, Flyweight, Mediator, Chain of Responsibility y Iterator/Visitor/Interpreter; Anexo B, sección 1.3) y señaló nueve idiomas que no son GoF ni GRASP, como la capa de servicios, el núcleo funcional con cáscara imperativa, los objetos de valor inmutables y las reglas guiadas por tablas (Anexo B, sección 1.4).

Tabla 4. Patrones GoF que el asistente ubica en el código (`claro` = el patrón está bien definido; `parcial` = cumple en parte; `discutible` = se parece pero no lo es en rigor)

Nº	Patrón	Descripción u objetivo	Dónde	Certeza	Verificado
C1	Simple Factory (variante)	Centraliza qué cliente de IA se crea según la configuración	`ai/__init__.py`, `get_ai_client()`	Parcial	Sí
C2	Simple Factory (banderas)	Construye objetos `Flag` inmutables desde una tabla de textos	`rules/flags.py`, `_flag`	Discutible	No
C3	Adapter	Traduce la API REST de Gemini a `AIResult` y a una jerarquía de errores propia	`ai/gemini.py`, `GeminiClient`	Claro	No
C4	Adapter (datos)	Convierte modelos del ORM en dataclasses congeladas para el motor de reglas	`services.py`, `exercise_items`, `food_items`; `health/services.py`, `build_facts`	Parcial	No

Nº	Patrón	Descripción u objetivo	Dónde	Certeza	Verificado
C5	Facade	Una llamada coordina ficha, banderas, elegibilidad y filtros de catálogo	`services.py`, `assess()`	Parcial	No
C6	Decorator (funcional)	Añade control de acceso y desactivación por interruptor a las vistas	`plans_feature_required`, `health_feature_required`	Parcial	No
C7	Strategy	Cliente de IA intercambiable (simulado o real) detrás de `AIClient`	`ai/base.py`, `ai/fake.py`, `ai/gemini.py`	Claro	Sí
C8	Strategy (tabla de despacho)	Elige el ejecutor del trabajo por tipo, sin `if/elif`	`jobs.py`, `_RUNNERS`	Parcial	Sí
C9	Template Method (funcional)	Esqueleto fijo de generar, validar y reintentar; los pasos variables llegan como parámetros	`jobs.py`, `_generate_validated`	Parcial	No
C10	Template Method (ganchos de Django)	Formularios, administración y comandos rellenan ganchos que define el framework	`admin.py`, `health/forms.py`, `management/commands/`	Claro (lo aporta Django)	No
C11	Command	El pedido de plan es un registro persistente que se encola, se ejecuta después y se reintenta	`models.py`, `PlanGenerationJob`; `jobs.py`	Parcial	No
C12	Observer (vía `on_commit`)	Callbacks que corren al confirmar la transacción (lanzar el hilo, enviar correo)	`services.py`, `nutrition.py`	Discutible	No
C13	State (implícito)	Ciclo de vida por estados con transiciones repartidas en condiciones	`Plan.status`, `PlanGenerationJob.status`	Discutible	Sí
C14	Memento (variante)	Versionado de la ficha y de las opciones de comida, con «deshacer» en estas últimas	`health/models.py`, `HealthProfile`; `MealOption.replaces`	Discutible	No

Tabla 5. Patrones GRASP que el asistente ubica en el código

Nº	Patrón	Descripción u objetivo	Dónde	Certeza	Verificado
G1	Information Expert	Los valores derivados los calcula el objeto que tiene los datos	`rules/eligibility.py`, `Eligibility`; `rules/flags.py`, `FlagReport`; modelos de `health`	Claro, con excepciones en las vistas	Sí (la regla de aprobación se repite en la vista)
G2	Creator	La función que recibe la salida validada crea rutina, plan, comidas y eventos	`services.create_exercise_draft`, `nutrition.create_nutrition_draft`	Parcial	No
G3	Controller	Vistas delgadas que delegan en servicios y traducen excepciones a mensajes	`views.trainer_plan_approve`, `services.request_plan`	Claro (tres vistas son gruesas)	No

Nº	Patrón	Descripción u objetivo	Dónde	Certeza	Verificado
G4	Low Coupling	Núcleo `rules/` sin Django; pero dependencia mutua entre `health` y `plans` e imports locales para evitar ciclos	`rules/*`; `services.py` líneas 288 y 506	Parcial	Parcial
G5	High Cohesion	`rules/` y `ai/` enfocados; los módulos de casos de uso agrupan varios temas	`services.py` (559 líneas), `nutrition.py` (520), `jobs.py` (479), `views.py` (670)	Parcial	Sí
G6	Polymorphism	El código trata igual a cualquier `AIClient`, sin condicionales por tipo	`jobs.py` líneas 332-333 y 436-438	Claro	Sí
G7	Pure Fabrication	Elementos artificiales que no son del negocio, creados para lograr cohesión	`jobs.py`, `notifications.py`, `rules/*`, `_SlotView`, `health/dates.py`	Claro	No
G8	Indirection	Intermediarios que permiten cambiar o simular lo que hay detrás	`get_ai_client()`, `dates.today()`, `adaptation.for_job`	Claro	No
G9	Protected Variations	Fronteras estables frente a lo que cambia: proveedor de IA, salida de la IA, reglas legales, modelo de datos	`AIClient`, `rules/validate.py`, `professional_required()`, `ExerciseItem`	Claro	No

4. Inventario de patrones omitidos

La Tabla 6 lista los patrones que el asistente dice que pudieron o debieron aplicarse. La columna *Valoración propuesta* es del equipo de trabajo asistido por IA: pondera el efecto de la ausencia sobre la facilidad de ampliar, de probar y de mantener, y **debe ser confirmada por el equipo**.

Tabla 6. Patrones omitidos, problema que resolverían y valoración de su relevancia

Nº	Patrón	Problema de diseño que resolvería	Valoración propuesta	Verificado
M1	State o tabla de transiciones	Las reglas de transición del plan están en 4 condiciones `status not in …` y en 10 creaciones manuales de `PlanEvent`; añadir un estado obliga a revisar varios sitios	**Media.** Una tabla de transiciones sí; clases State, no (Sección 6)	Sí
M2	Template Method o cierre común de los ejecutores	El bloque de cierre (bloqueo de fila, `lost_lock`, creación, cierre del trabajo) está copiado tres veces en `jobs.py`	**Alta.** Código de concurrencia duplicado: un arreglo debe hacerse en tres sitios	Sí
M3	Template Method o fábrica común para crear trabajos	`request_plan` y `request_alternative` repiten el flujo y ya divergieron: solo la primera evita repetir el aviso de bloqueo	**Alta.** Posible defecto latente, según la lectura del código (no se ejecutó): cada nuevo pedido de un caso bloqueado crea otro trabajo y otro correo	Sí

Nº	Patrón	Problema de diseño que resolvería	Valoración propuesta	Verificado
M4	Null Object (adaptación)	`adaptation` puede ser `None` y 10 líneas de `jobs.py` lo comprueban	**Media.** Menos ramas y pruebas más simples	Sí
M5	Strategy por tipo de plan	Las comparaciones por tipo (ejercicio, alimentación, alternativa) están repartidas en cinco módulos	**Baja por ahora** (solo hay dos tipos); sube si se agrega un tercero	Parcial (cifra corregida)
M6	Specification para «¿se puede aprobar?»	La regla está en la vista, en `approve_plan` y en `nutrition.approval_problems`; si una cambia, la interfaz puede mostrar un botón que el servicio rechaza	**Media.**	Sí
M7	Facade o consulta única de «alimentos y ejercicios permitidos»	La composición del filtro se repite en tres sitios y corre hasta tres veces en una pantalla	**Media-baja.**	No
M8	Decorator o Proxy sobre `AIClient`	La política de reintentos está mezclada con el presupuesto de tiempo y los latidos	**Baja** (hay un solo proveedor)	No
M9	Objetos de regla para las banderas de seguridad	`flags.evaluate` es una función de unas 120 líneas con 44 condiciones	**Baja hoy;** media cuando un profesional cambie los umbrales	No
M10	Observer o eventos de dominio propios	Los avisos y la auditoría se llaman directamente; otro canal obligaría a tocar cada sitio	**Baja** (un solo canal)	No
M11	Inyección de dependencias del cliente de IA	Tres ejecutores llaman a `get_ai_client()` y las pruebas deben parchearlo	**Baja** (`mock.patch` ya funciona)	No
M12	Builder o DTO tipado del payload a la IA	La lista de lo único que viaja a la IA es un `dict` armado a mano y repetido en el simulado y en `check_gemini`	**Media** (frontera de privacidad)	No
M13	Repository o `QuerySet` propio	13 consultas `PlanGenerationJob.objects.filter(pk=…)` en `jobs.py`; no existe capa de repositorio	**Baja.** Un `QuerySet` sí; una capa Repository formal, no	Sí (13)
M14	Pipeline o Chain of Responsibility para validar la salida	Validadores largos pero cohesivos	**Nula:** añadiría indirección sin variación real	No
M15	Strategy para los cupos de uso	Cuatro funciones repiten «contar trabajos en una ventana de tiempo»	**Nula como patrón:** basta extraer un método	No

5. Clasificación: indicado o espontáneo

Para cada patrón aplicado se determinó si el material que guió la generación (Tabla 2) lo pedía o lo describía como decisión estructural (*indicado*), si lo pedía solo en parte (*mixto*) o si lo incorporó el asistente por sí solo (*espontáneo*). Como el prompt original no se conserva, la evidencia es el contrato `plan-ia.md` y las pautas del agente; para fechar el origen de algunos elementos se usó el historial de git.

Tabla 7. Origen de cada patrón aplicado

Nº	Patrón	Origen	Evidencia
C1	Simple Factory	Indicado	Contrato §4.3: «se inyecta con `get_ai_client()`»
C2	Factory de banderas	Espontáneo	Sin mención en el contrato
C3	Adapter (Gemini)	Indicado	Contrato §4.3: interfaz `AIClient` y errores normalizados
C4	Adapter (datos)	Espontáneo	El contrato pide reglas puras (§3), no la forma de las dataclasses
C5	Facade	Espontáneo	El contrato describe la salida `Eligibility`, no la fachada `assess`; aparece en la fase B (`42ed7e0`)
C6	Decorator	Espontáneo	Idioma de Django; el proyecto ya usaba decoradores como `member_required`
C7	Strategy (`AIClient`)	Indicado	Contrato §1: «IA detrás de una interfaz»; §4.3: modos `simulated` y `live`
C8	Tabla de despacho	Espontáneo	Aparece en la fase C (`925d7f0`), al sumar el tercer tipo de trabajo
C9	Template Method funcional	Espontáneo	Aparece en la fase C (`925d7f0`), al compartir la generación entre tres casos
C10	Template Method (Django)	Espontáneo	Lo aporta el framework; el proyecto solo rellena los ganchos
C11	Command	Indicado	Contrato §6.1 y §8.1: cola en base de datos y `PlanGenerationJob`
C12	Observer (`on_commit`)	Indicado	Contrato §6.1: «en `transaction.on_commit`, lanza un hilo»
C13	State (implícito)	Indicado solo en prosa	Contrato §8.1 lista estados y transiciones en texto; no pide el patrón
C14	Memento (versionado)	Indicado	Contrato §1: «Filas versionadas»; `Plan.version` en §8.1
G1	Information Expert	Espontáneo	No aparece en el contrato ni en las pautas
G2	Creator	Espontáneo	No aparece en el contrato ni en las pautas
G3	Controller	Indicado	Pauta «separa controladores y servicios» y contrato §1
G4	Low Coupling	Mixto	El contrato fija «reglas puras sin Django ni red»; el acoplamiento restante no se planeó

Nº	Patrón	Origen	Evidencia
G5	High Cohesion	Mixto	Pauta «separa responsabilidades»; el tamaño de los módulos de servicio no se acordó
G6	Polymorphism	Indicado	Consecuencia de la interfaz `AIClient` (C7)
G7	Pure Fabrication	Mixto	`jobs.py` y `notifications.py` salen del contrato; `_SlotView` y `dates.py` no
G8	Indirection	Mixto	`get_ai_client()` está en el contrato; `dates.today()` no
G9	Protected Variations	Mixto	Los modos `AI_PLANS_MODE` salen del contrato (§4.3); `GEMINI_SCHEMA_FIELD` y `professional_required()` se decidieron al construir

Tabla 8. Resumen de la clasificación

Origen	Cantidad	Patrones
Indicado	9	C1, C3, C7, C11, C12, C13, C14, G3, G6
Mixto	5	G4, G5, G7, G8, G9
Espontáneo	9	C2, C4, C5, C6, C8, C9, C10, G1, G2
Total	**23**	

5.1 Qué tipo de patrones aplica la IA por sí sola y cuáles omite

1. **Lo espontáneo es de pequeña escala y de estilo funcional**: fachadas, adaptadores de datos, decoradores, tablas de despacho, un Template Method con funciones y objetos de valor inmutables, además de asignaciones de responsabilidad (Experto, Creador). Casi nunca son jerarquías de clases de manual: la única jerarquía propia con polimorfismo es la de `AIClient`, que el contrato pedía.
2. **Los patrones aparecen cuando aparece la duplicación real.** La tabla de despacho `_RUNNERS` y el Template Method `_generate_validated` no existían en la fase B; nacieron en la fase C, al llegar el tercer caso similar (commit `925d7f0`). Es la regla de tres aplicada por el asistente sin que nadie se lo pidiera.
3. **Donde el contrato dio la decisión estructural, el código la siguió con fidelidad**: interfaz del cliente de IA, cola en base de datos, `on_commit`, versionado. Las filas «indicadas» corresponden sobre todo a decisiones de frontera entre módulos.
4. **Lo omitido es de tres clases.** (a) *Consolidar código ya escrito* (M2, M3, M6, M7): el asistente aplica patrones al escribir lo nuevo, pero rara vez vuelve a unificar copias anteriores; la divergencia de M3 es la consecuencia. (b) *Abstracciones sin presión inmediata* (State, Null Object, Strategy por tipo): con dos tipos de plan y pocas transiciones, el código con condiciones funciona. (c) *Pautas genéricas con nombre propio*: la pauta pedía «repositorios (acceso a datos)» y no hay ninguna capa Repository; el asistente adaptó la pauta a las convenciones de Django (modelos y `QuerySet`) sin dejar constancia de la desviación.

6. Experimento de reformulación

6.1 Pregunta y diseño

El asistente señaló como omisión relevante el patrón State para el ciclo de vida del plan (M1), aunque aclaró que «clases State completas: no» valían la pena y que bastaba una tabla de transiciones. El experimento pone a prueba ese juicio. Pregunta: *si se pide State de forma explícita, ¿lo incorpora el asistente y mejora la facilidad de ampliar y de probar el módulo?*

Módulo. Los cambios de estado de un `Plan` (tomar para revisión, aprobar, archivar y regenerar), unas 100 líneas del original (`services.py`, líneas 462 a 559). Se comparan tres versiones:

- **Original (V0):** el código del repositorio.
- **A, control:** el módulo regenerado con la especificación reconstruida y las pautas genéricas de arquitectura, sin nombrar patrones (Anexo C). Se incluye para separar el efecto del prompt del efecto de regenerar el código en otra sesión.
- **B, reformulado:** el mismo prompt que A más un único párrafo que pide el patrón State (GoF) con sus consecuencias. La única diferencia entre los dos prompts, verificada con `diff`, es ese párrafo y el nombre de la carpeta de salida (Anexo C).

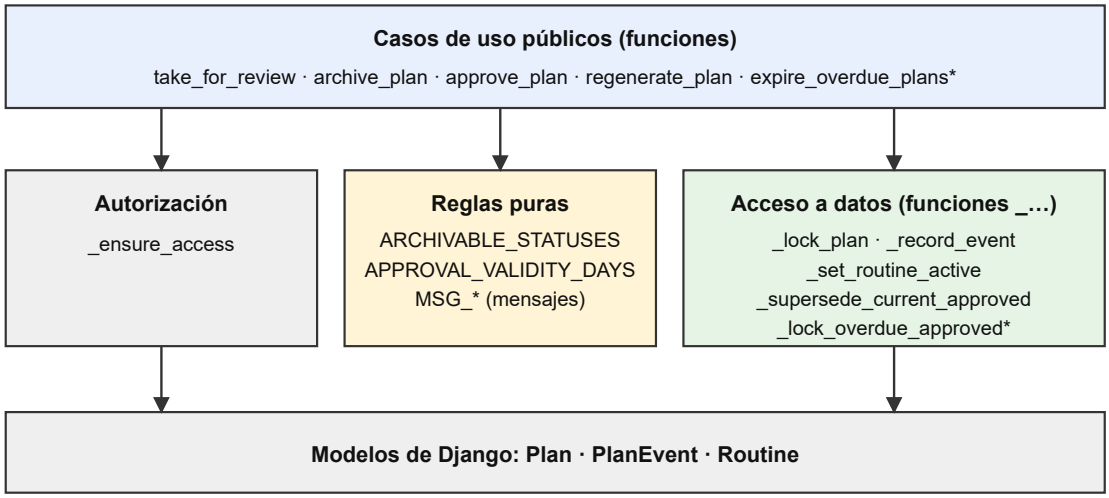
Misma herramienta y mismo modelo. A y B se generaron con Claude Code y el modelo `claude-sonnet-5-5`, cada uno en una conversación nueva. Se les prohibió leer las funciones de ciclo de vida existentes para que no copiaran el original. El código original lo escribieron en parte Opus 5.5 (fase B) y en parte Sonnet 5.5 (fase C); la comparación con V0 es, por tanto, solo de referencia.

Medida de facilidad de ampliar. A cada versión se le pidió, en una conversación nueva y con el mismo texto, un cambio realista: agregar el estado «vencido» (`expired`) para planes aprobados con vigencia pasada, con una operación nueva `expire_overdue_plans` y reglas sobre qué se puede hacer con un plan vencido (Anexo C). Se midió el diff, los lugares del código tocados y las pruebas necesarias. El equipo corrió todas las pruebas por su cuenta.

6.2 Resultados

La Figura 1 muestra la estructura de A y la Figura 2 la de B. Ambas versiones respetan las firmas, las reglas y los mensajes pedidos y pasan sus pruebas (A, 45; B, 51).

Figura 1. Estructura del módulo A (control): capas con funciones



* añadido en la ampliación (estado «vencido»). Las flechas indican «usa».

Figura 2. Estructura del módulo B (reformulado): patrón State con contexto y repositorio

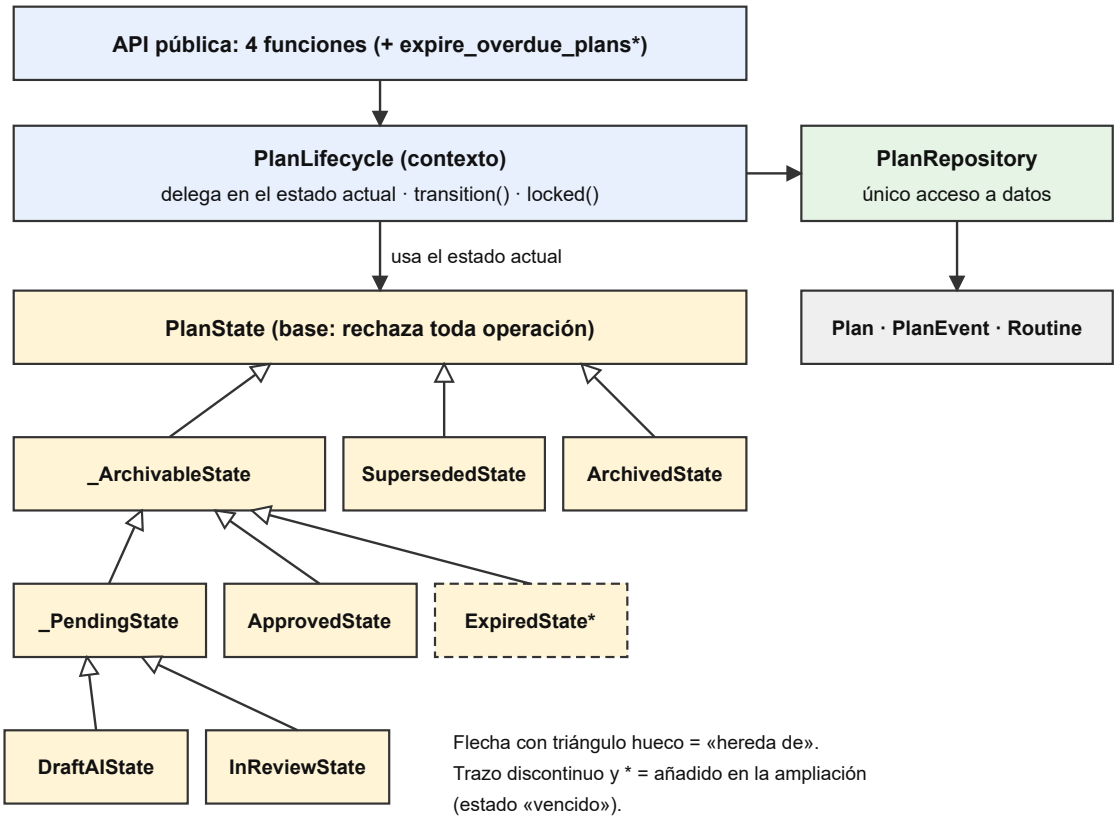


Tabla 9. Estructura de las tres versiones antes de ampliar

Métrica	V0 original	A control	B con State
Líneas del módulo (incluye comentarios)	98	227	326

Métrica	V0 original	A control	B con State
Funciones o métodos	5	12	38
Clases	0	0	10
Condicionales `if`/`elif`	18	13	10
Dónde se decide qué transiciones son válidas	4 condiciones `status not in …` repartidas en 3 funciones (`approve_plan` tiene dos)	1 tupla (`ARCHIVABLE_STATUSES`) y la propiedad `is_pending`	Cada clase de estado, por herencia y sobrescritura
Pruebas del módulo	No comparable (están en la suite del proyecto)	45	51
Pruebas que no usan base de datos	—	0	3 (registro de estados)

Tabla 10. Costo de agregar el estado «vencido» (cambio idéntico pedido a A y a B)

Métrica	A control	B con State
Líneas del módulo añadidas y eliminadas	+34 y −5	+47 y −9
Lugares de código tocados (sin docstrings)	5: constantes, tupla `ARCHIVABLE_STATUSES`, un auxiliar, la función pública, `__all__`	11: constantes, consulta del repositorio, método base `expire`, `ApprovedState.expire`, clase `ExpiredState`, registro `_STATES`, `PlanLifecycle.expire`, dos líneas de registro (`transition` y `record`), la función pública, `__all__`
Operaciones existentes que no hubo que modificar	4 (`take_for_review`, `approve_plan`, `regenerate_plan` y el reemplazo del aprobado anterior)	Las mismas 4, por la clase base que rechaza todo
Pruebas añadidas	22 (de 45 a 67)	22 (de 51 a 73)
Resultado de las pruebas	67 OK	73 OK

Aunque no se ejecutó, la lectura del original sugiere que la misma ampliación en V0 tocaría la condición de `archive_plan` y añadiría la función nueva: las otras tres operaciones comparan contra `PENDING_STATUSES` y ya rechazarían el estado nuevo. Es una inferencia, no una medida.

6.3 Comparación argumentada

Al pedirlo, el asistente incorporó el patrón. B tiene una clase base `PlanState` que rechaza todo, cinco estados concretos que sobrescriben solo lo que admiten y un contexto `PlanLifecycle` que delega, sin `if plan.status` en las operaciones públicas. Mantuvo las firmas y los mensajes (Gamma et al., 1994, describen esta estructura).

Ventajas de B. (1) Los efectos de entrar en un estado (evento que se registra y activación o desactivación de la rutina) se declaran una sola vez por clase y los aplica una única función `transition`; en A, cada operación repite a mano los tres pasos de cambiar el estado, ajustar la rutina y registrar el evento, y la función nueva `expire_overdue_plans` los repitió. (2) Un estado nuevo hereda por defecto el rechazo de todo, y `ExpiredState` obtuvo `archive` gratis por herencia. (3) El patrón dejó el registro de estados verificable sin base de datos (3 pruebas).

Costos de B. Tiene un 44 % más de líneas que A (326 contra 227), 38 funciones y 10 clases contra 12 funciones, y exigió 11 lugares de código para la ampliación contra 5. Para entender una operación hay que recorrer varias clases de la jerarquía (por ejemplo, `approve` está en `_PendingState` y no en `DraftAIState`). Las pruebas siguen necesitando base de datos casi en su totalidad: el repositorio es inyectable, pero según una búsqueda en el archivo ninguna prueba lo reemplaza por un doble.

Dónde el patrón no mejora el diseño. En este módulo los estados casi no se diferencian por comportamiento: casi todas las operaciones se permiten en los mismos dos estados. Por eso A ya conseguía la ventaja principal de State, la ausencia de condicionales dispersos, con una tupla de estados archivables y la propiedad `is_pending` (una lista de permitidos funciona como tabla de transiciones). Con cinco o seis estados y reglas parecidas, la jerarquía añade conceptos sin quitar trabajo al ampliar. State pagaría cuando cada estado tenga reglas propias distintas por operación, o cuando haya muchos más estados. Esto confirma el juicio del asistente de la consulta (tabla sí, clases no).

Precaución sobre la comparación. B también creó una clase `PlanRepository` y A usó funciones auxiliares, aunque las dos recibieron la misma pauta genérica de «separar acceso a datos». Esa diferencia no se debe al párrafo de State: es variabilidad entre corridas. Por eso no se declara superior a B ni a A: A resultó más simple de ampliar y B centralizó los efectos de cada estado.

7. Conclusiones y recomendación

7.1 Conclusiones

1. **Qué hizo el asistente de forma espontánea.** Aplicó patrones ligeros y funcionales (fachada, adaptadores, decoradores, tablas de despacho, Template Method con funciones) y asignó responsabilidades con criterio GRASP. Los patrones aparecieron al repetirse el caso, no por anticipado (Tabla 7 y Sección 5.1).
2. **Qué omitió.** Unificar código duplicado que ya diverge (M2, M3), el patrón State y Null Object, y una capa de repositorios que la pauta pedía por su nombre. La omisión

más costosa no fue de un patrón de catálogo: fue la duplicación entre `request_plan` y `request_alternative`.

3. **Qué cambió al indicar el patrón en el prompt.** El asistente lo incorporó y conservó el comportamiento, pero el módulo creció un 44 %, y ampliarlo costó más lugares de código (11 contra 5) sin una ventaja clara para este tamaño de problema.
4. **Qué recomendar.** No pedir nombres de patrones como lista de verificación. Pedir *principios con su límite* (una sola fuente de verdad, transiciones en una tabla, interfaz solo con dos implementaciones reales), una pasada de consolidación de duplicación y una auditoría final en la que el asistente liste lo que aplicó y lo que descartó.

7.2 Qué indicar en un prompt de generación, cuándo y cómo

Tabla 11. Recomendaciones para el prompt de generación

Situación	Qué indicar	Qué no indicar
Fronteras entre módulos (proveedor externo, cola, versionado)	La decisión estructural y la interfaz, como hizo el contrato (el código la siguió con fidelidad)	El nombre del patrón sin el problema que resuelve
Ciclo de vida con pocas transiciones	Las transiciones en una tabla única y una función que aplica una transición	«Aplica State»: genera una jerarquía que este tamaño no justifica
Variación entre implementaciones	Una interfaz común solo si hoy existen al menos dos implementaciones reales	Interfaces «por si acaso»
Duplicación	Extraer el bloque que se repita tres veces y unificar copias que ya difieren, con una prueba que fije el comportamiento correcto	Esperar que el asistente consolide solo
Capas que el framework ya resuelve	Decir explícitamente qué usar (en Django, modelos y `QuerySet` en lugar de un Repository)	«Separa repositorios» sin aclarar el contexto del framework
Cierre de la generación	Pedir la lista de patrones y principios aplicados y de los considerados y descartados, con su razón	Medir la calidad por la cantidad de patrones

7.3 Versión mejorada del prompt de generación

```
ROL Y CONTEXTO
Eres un desarrollador senior. Trabajas en <proyecto y framework>.

QUÉ CONSTRUIR
<Especificación funcional: operaciones con sus firmas, estados, reglas y mensajes de
error.>

ARQUITECTURA
- Capas: vistas o controladores delgados -> servicios (un caso de uso por función) ->
  modelos y consultas del framework. No crees una capa Repository si el framework ya
ofrece
  Manager o QuerySet; si la creas, justifícalo en una línea.
- Las reglas de negocio puras (sin framework ni red) van en un módulo aparte.

PRINCIPIOS DE DISEÑO (se pide el principio y su límite, no el nombre de un patrón)
1. Una sola fuente de verdad por regla: si la misma regla se evalúa en dos lugares
```

```
(por ejemplo
   vista y servicio), el servicio la expone y la vista la consulta.
2. Ciclo de vida: define las transiciones permitidas en una sola tabla (estado actual,
   operación, estado destino) y una única función que aplica una transición (valida,
guarda,
   ajusta los efectos y registra el evento). Usa una clase por estado solo si cada
estado tiene
   reglas propias distintas.
3. Variación: usa una interfaz común únicamente si existen hoy al menos dos
implementaciones.
4. Valores ausentes: si varios sitios comprueban None, devuelve un objeto vacío.
5. Duplicación: si un bloque se repite en tres lugares, extráelo. Si dos copias de un
mismo
   flujo ya difieren, unifícalas y deja una prueba que fije el comportamiento
correcto.

LÍMITES
- No agregues jerarquías, fábricas ni capas que la especificación no necesite. Cada
abstracción
  nueva se justifica en una línea con el problema concreto que resuelve.

ENTREGA
- Código y pruebas. Al final, lista los patrones o principios aplicados (dónde y por
qué) y los
  que consideraste y descartaste (por qué).
```

7.4 Limitaciones del experimento

- **Un solo asistente y una sola familia de modelos.** La consulta, la generación y la ampliación las hizo Claude Sonnet 5.5; el código original lo escribieron Sonnet 4.6, Sonnet 5.5 y Opus 5.5. Es una IA evaluando código de su misma familia, sin contraste con otro asistente.
- **Un solo código y un solo módulo.** Las conclusiones sobre State valen para un ciclo de vida de cinco estados y pocas diferencias de comportamiento; con otro módulo podrían ser distintas.
- **Una sola corrida por variante.** No se repitió cada generación para estimar la variabilidad. Que B creara `PlanRepository` y A no, con la misma pauta, muestra que existe y que las diferencias de tamaño no se pueden atribuir enteras al párrafo de State.
- **Prompt original reconstruido.** El control A usa una reconstrucción, no el texto real; la clasificación del paso 4 se basa en el contrato y en las pautas, no en los mensajes originales.
- **Conversación «nueva» con reservas.** No se pudo comprobar que el agente de consulta no recibiera `CLAUDE.md` como contexto automático; se le indicó no usarlo. Tampoco se verificó independientemente el modelo que cada agente declara ser.
- **Imprecisiones de la consulta.** El asistente dio una cifra que no se reproduce (comparaciones por tipo) y no se verificaron todas sus afirmaciones (Tablas 4 a 6, columna *Verificado*).

- **Métricas estructurales, no de calidad.** Se midieron líneas, clases y lugares tocados; no se evaluó la corrección fina ni el desempeño del código. La valoración de relevancia de la Tabla 6 es una propuesta pendiente de confirmación del equipo.

7.5 Hallazgo práctico fuera del alcance de la actividad

La verificación de M3 reveló un posible defecto latente en el sistema real. Según la lectura del código (no se ejecutó), en `nutrition.request_alternative` un caso bloqueado por bandera roja no evita crear un trabajo y un aviso nuevos en cada pedido con un token distinto, y los trabajos bloqueados no consumen cupo; `services.request_plan`, en cambio, deduplica el aviso por día. Conviene confirmarlo con una prueba y corregirlo en un cambio aparte.

8. Registro de uso de IA

Tabla 12. Herramientas de IA utilizadas

Herramienta	Para qué se usó	Versión del modelo
Claude Code (sesión principal)	Dirigir el experimento, verificar las respuestas contra el código, ejecutar pruebas, medir diffs y redactar este informe	`claude-sonnet-5-5`
Claude Code, agente de consulta	Consulta de patrones aplicados y omitidos (Anexos A y B)	`claude-sonnet-5-5` (declarado por el agente)
Claude Code, 2 agentes de generación	Generar los módulos A (control) y B (con State)	`claude-sonnet-5-5` (declarado)
Claude Code, 2 agentes de ampliación	Agregar el estado «vencido» a A y a B	`claude-sonnet-5-5` (declarado)
Claude Code (código original del sistema)	Generación de ProFit Studio entre mayo y octubre de 2026	Sonnet 4.6, Sonnet 5.5 y Opus 5.5 (firmas de los commits)

Herramientas que no son de IA: `git`, el ejecutor de pruebas de Django y Chrome en modo sin interfaz para generar el PDF.

Tabla 13. Evolución de los prompts principales

Prompt	Versión	Cambio respecto de la anterior	Motivo
Consulta	1	Texto inicial; no hubo iteraciones	Cubrir el paso 2 de la actividad sin forzar patrones
Generación (A, control)	1.0	Reconstrucción del original: especificación del ciclo de vida más las pautas de arquitectura del agente	El prompt original no se conserva
Generación (A, control)	1.1	Se quitó la nota inicial que explicaba el experimento	Evitar que el agente supiera que se comparaban patrones

Prompt	Versión	Cambio respecto de la anterior	Motivo
Reformulado (B)	2	Se agregó un párrafo que pide State (GoF); el resto es idéntico (verificado con `diff`)	Patrón omitido M1 de la Tabla 6
Ampliación (A y B)	1	Mismo texto para ambas; solo cambia la carpeta (verificado con `diff`)	Medir la facilidad de ampliar

Los informes completos de cada agente no se reproducen; lo que importa para las conclusiones está en los anexos y el equipo corrió por su cuenta todas las pruebas citadas.

9. Material de consulta

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.ª ed.). Prentice Hall.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

Repositorio del código: `https://github.com/Jdarenas22/profit-studio`. El experimento está en la carpeta `docs/actividad-patrones/`.

Anexo A. Prompt de consulta (íntegro)

Texto exacto enviado al asistente en una conversación nueva (archivo `anexos/A-prompt-de-consulta.md`). En todos los prompts de los anexos, la ruta absoluta del equipo local se sustituyó por `<RAIZ_DEL_PROYECTO>` antes de publicar el informe; el resto del texto es íntegro.

```
Eres un asistente de IA que revisa código. Esta es una conversación nueva: no tienes
historial previo sobre cómo se generó este código y no debes buscarlo.

CONTEXTO
El proyecto Django está en <RAIZ_DEL_PROYECTO>. Analiza únicamente el código de
producción de dos apps: `apps/plans` y `apps/health` (excluye `tests/`, `migrations/`
y `templates/`). Es una aplicación web de gestión de un gimnasio; esas dos apps
implementan la ficha de salud de la clienta y los planes de ejercicio y de
alimentación generados con IA.

REGLAS DE LA CONSULTA
- Solo lectura: no modifiques ni crees archivos del proyecto, excepto el archivo de
respuesta indicado abajo.
- Basa tus respuestas únicamente en el código. No leas `CLAUDE.md`, ni nada de la
carpeta `docs/`, ni el historial de git: queremos tu análisis independiente de lo que
digan los documentos.
- No fuerces patrones: lista un patrón solo si puedes señalar el lugar concreto del
código (archivo y clase o función, con número de línea aproximado) y explicar por qué
lo es. Si es parcial o discutible, dilo. Si el patrón lo aporta el framework (Django)
y no el código del proyecto, dilo.

PREGUNTAS
1. ¿Qué patrones de diseño GoF (creacionales, estructurales y de comportamiento) y qué
patrones GRASP contiene este código? Para cada uno: nombre, familia (GoF creacional,
estructural o de comportamiento; o GRASP), descripción de su intención, dónde está
(archivo, clase o función, línea aproximada), evidencia breve y grado de certeza
(claro, parcial o discutible).
2. ¿Qué patrones de diseño pudo o debió aplicar y no aplicó? Para cada uno: nombre, el
problema de diseño concreto que resolvería (con un ejemplo tomado del código), dónde
se aplicaría, qué mejoraría en la facilidad de ampliar, de probar o de mantener, el
costo o el riesgo de aplicarlo y si en tu opinión valdría la pena (sí, no o quizá, con
la razón).
3. Al final, un resumen de 5 a 8 líneas sobre qué tipo de patrones predominan y cuáles
faltan.

FORMATO
Responde en español, con tablas Markdown para los dos inventarios. En la primera línea
indica el ID exacto del modelo que eres. Guarda tu respuesta completa, exactamente
como la entregas, en `docs/actividad-patrones/anexos/B-respuesta-del-asistente.md`. Tu
mensaje final debe decir solo la ruta del archivo guardado y cuántas líneas tiene, sin
repetir la respuesta.
```

Anexo B. Respuesta íntegra del asistente

Respuesta tal como la guardó el asistente (archivo `anexos/B-respuesta-del-asistente.md`), convertida de Markdown a HTML sin cambiar su contenido.

Modelo: claude-sonnet-5-5

Patrones de diseño en `apps/plans` y `apps/health` (código de producción)

Alcance y método. Leí los 46 archivos `.py` de producción de las dos apps (unas 8 200 líneas; sin `tests/`, `migrations/` ni `templates/`). Es un análisis estático: no ejecuté el código ni vi las pruebas. Los números de línea son aproximados (`~L`). Cuando un patrón lo aporta Django y no el proyecto, lo indico. Las rutas son relativas a la raíz del proyecto.

---

1. Patrones presentes

1.1 GoF (creacionales, estructurales y de comportamiento)

Nº	Patrón	Familia	Intención	Dónde	Evidencia	Certeza
C1	Simple Factory (variante; no es el Factory Method puro)	GoF creacional	Centralizar la decisión de qué objeto concreto se crea según la configuración.	`apps/plans/ai/__init__.py`, `get_ai_client()` ~L40-52, apoyada en `ai_mode()` ~L23-28 y `live_is_configured()` ~L35-37.	Según `AI_PLANS_MODE` devuelve `FakeAIClient()` o `GeminiClient(...)` (imports diferidos) o lanza `AIPermanentError('ai_not_configured')`. `jobs._run_exercise` ~L314 y `_run_nutrition` ~L411 solo piden "un cliente". No hay clase creadora con subclases que redefinan un método fábrica; por eso no es el Factory Method de GoF.	Parcial
C2	Simple Factory (fábrica de banderas)	GoF creacional (variante)	Encapsular los detalles de construcción de un objeto.	`apps/plans/rules/flags.py`, `_flag`, `_red`, `_yellow` ~L248-259.	`_flag` busca texto y acción en `_TEXTS` y construye el `Flag` inmutable; `evaluate` (~L272-390) nunca llama `Flag(...)` directamente. No hay variación de tipos, solo de datos.	Discutible
C3	Adapter	GoF estructural	Convertir la interfaz de un servicio externo a la que espera el cliente.	`apps/plans/ai/gemini.py`, `GeminiClient` ~L42-138 (`generate_json` ~L73-102, `parse_response` ~L104-138), que implementa `AIClient` (`apps/plans/ai/base.py` ~L57-62).	Traduce la API REST de Gemini (`candidates`, `usageMetadata`, `finishReason`, códigos HTTP, timeouts) a `AIResult`/`AIUsage` y a la jerarquía `AITransientError`/`AIPermanentError`/`AIBlockedError`/`AIInvalidOutput` con códigos como `ai_http_429`. El resto del sistema no conoce el formato de Google.	Claro
C4	Adapter (capa anticorrupción de datos)	GoF estructural (variante)	Aislar el dominio de la forma de los datos de otra capa.	`apps/plans/services.py`: `exercise_items` ~L141-152, `food_items` ~L163-179, `training_profile` ~L155, `nutrition_profile` ~L182. `apps/health/servic`	Convierten `Exercise`, `Food`, `HealthProfile` y consentimientos del ORM en dataclasses congeladas (`ExerciseItem`, `FoodItem`, `Facts`) que consumen los módulos de `rules/` sin importar Django. Son funciones de mapeo, no clases que implementen una interfaz destino.	Parcial

Nº	Patrón	Familia	Intención	Dónde	Evidencia	Certeza
				es.py: build_facts ~L414-451.		
C5	Facade	GoF estructural	Dar una interfaz única y simple a un subsistema.	apps/plans/services.py, assess() ~L205-235, que devuelve Assessment (~L122-138).	Una sola llamada coordina health.current_profile, profile_flags, rules_eligibility.evaluate, rules_catalog.filter_exercises, rules_foods.filter_foods y energy_and_slots. La usan las vistas, jobs._evaluate_for_run (~L277) y nutrition.edit_option (~L231) sin conocer las reglas. Es una función, no una clase, y ramifica por kind. El paquete apps/plans/ai/__init__.py actúa de fachada menor al reexportar nombres de base.py.	Parcial
C6	Decorator (forma funcional de Python) con un toque de Chain of Responsibility	GoF estructural (y de comportamiento)	Añadir una responsabilidad a una función sin modificarla.	apps/health/decorators.py, health_feature_required ~L9-17. apps/plans/views.py, plans_feature_required ~L50-58. Se apilan como @plans_feature_required @member_required @require_GET (por ejemplo member_plans ~L111-114).	Envuelven la vista con functools.wraps, responden 404 si la función está apagada y añaden never_cache. Cada decorador puede cortar la cadena (404, 403, 405). Es el mecanismo de decoradores del lenguaje y de Django, no el GoF Decorator por composición de objetos.	Parcial
C7	Strategy	GoF de comportamiento	Familia de algoritmos intercambiables detrás de una misma interfaz.	AIClient (apps/plans/ai/base.py ~L57-62), FakeAIClient (ai/fake.py ~L140-197), GeminiClient (ai/gemini.py ~L42); elegidos por get_ai_client.	Las dos clases implementan generate_json(*, task, system, payload, schema, job_ref) y exponen model_name e is_simulated. jobs.py ~L333 y ~L437 usa client.is_simulated y client.model_name sin preguntar el tipo; el modo (off, simulated, live) decide en ejecución cuál se usa.	Claro
C8	Strategy en forma de tabla de despacho	GoF de comportamiento (variante funcional)	Elegir el comportamiento por clave sin if/elif.	apps/plans/jobs.py, _RUNNERS ~L475-479, usado en run_job ~L146.	Mapea KIND_EXERCISE, KIND_NUTRITION y nutrition.ALTERNATIVE a _run_exercise, _run_nutrition y _run_alternative (misma firma (job, started)). Son funciones, no objetos con interfaz común.	Parcial
C9	Template Method (funcional, por funciones de orden superior)	GoF de comportamiento (variante)	Fijar el esqueleto de un algoritmo y dejar pasos variables.	apps/plans/jobs.py, _generate_validated ~L250-274; la usan _run_exercise ~L319 y _generate_slot_options ~L395.	El esqueleto (armar payload, llamar con reintentos, acumular tokens, validar, un reintento enviando solo los códigos del error) es fijo; los pasos variables llegan como make_payload, make_schema y validate. Se logra con parámetros, no con herencia.	Parcial
C10	Template Method (ganchos de Django)	GoF de comportamiento	Esqueleto fijo con ganchos que redefine el cliente.	apps/plans/admin.py, SuperuserOnlyAdmin ~L12-29 (redefine has_*_permission de ModelAdmin). apps/health/forms.py, HealthProfileForm.clean_birth_date, clean_clearance_da	El esqueleto lo controla Django (full_clean, ModelAdmin, BaseCommand); el proyecto rellena los ganchos. SuperuserOnlyAdmin sí es una base propia que reutilizan TrainerCredentialAdmin y FoodAdmin. El patrón lo aporta el framework.	Claro, pero aportado por Django

Nº	Patrón	Familia	Intención	Dónde	Evidencia	Certeza
				te, clean ~L218-328. Command.handle en apps/plans/management/commands/.		
C11	Command (pedido persistente en cola)	GoF de comportamiento	Encapsular un pedido como objeto para encolarlo, ejecutarlo después y registrarlo.	apps/plans/models.py, PlanGenerationJob ~L130-221. apps/plans/jobs.py: start_job ~L62-68, claim ~L81-90, run_job ~L138-167, recover_stale_jobs ~L93-120. Se crea en services.request_plan ~L311-330.	El trabajo guarda tipo, clienta, notas, request_token, estado e intentos; se ejecuta después en un hilo y puede relanzarse. Pero no tiene un método execute(): los datos están en el modelo y la lógica en jobs.py (invocador) y _RUNNERS (receptores).	Parcial
C12	Observer (vía transaction.on_commit)	GoF de comportamiento	Avisar a interesados cuando ocurre un evento.	apps/plans/services.py ~L316, ~L329, ~L544; apps/plans/nutrition.py ~L454, ~L468.	Registran un callback que corre al confirmarse la transacción (lanzar el hilo, enviar correo). Lo provee Django; no hay sujeto con lista de observadores ni señales propias (la búsqueda de receiver y signals en ambas apps no encontró nada). Las notificaciones de jobs.run_job ~L150-166 son llamadas directas.	Discutible
C13	State (solo como máquina de estados implícita)	GoF de comportamiento (no aplicado como clases)	Cambiar el comportamiento según el estado del objeto.	Plan.status (apps/plans/models.py ~L37-127) y PlanGenerationJob.status (~L130-221). Transiciones en services.take_for_review ~L462, _archive ~L473, approve_plan ~L494, jobs.claim ~L81, _finish ~L128 y recover_stale_jobs ~L93.	Hay estados con nombre, grupos (PENDING_STATUSES, ACTIVE_STATUSES, FINAL_STATUSES) y transiciones protegidas con UPDATE condicional (filter(status=...).update(...)). No hay clases de estado ni tabla de transiciones: el ciclo de vida está repartido en condiciones. Es una máquina de estados, no el patrón State.	Discutible
C14	Memento (versionado persistente, sin restauración general)	GoF de comportamiento (variante)	Guardar el estado para poder volver atrás.	apps/health/models.py, HealthProfile ~L135-294 y apps/health/services.save_profile ~L275-323; ConsentTextVersion ~L45-94; Plan.version y Plan.parent. MealOption.replaces (apps/plans/models.py ~L392) con nutrition.review_option(action='revert') ~L258-282.	Cada guardado de la ficha crea una fila nueva con version y una sola is_current; las opciones de comida reemplazadas se pueden recuperar. Es historial de versiones, sin originador/caretaker; solo las opciones de comida tienen "deshacer".	Discutible

1.2 GRASP

Nº	Patrón	Familia	Intención	Dónde	Evidencia	Certeza
G1	Information Expert	GRASP	Asignar la responsabilidad a quien tiene la información.	apps/plans/rules/eligibility.py, Eligibility ~L59-81. rules/flags.py,	Los valores derivados (blocked, can_request, conservative, enough, can_generate, age) los calcula el objeto que tiene los datos.	Claro en rules/ y

Nº	Patrón	Familia	Intención	Dónde	Evidencia	Certeza
				`FlagReport` ~L75-105. `rules/foods.py`, `FoodCatalog` ~L91-114 y `Totals.fat_share` ~L166. `rules/catalog.py`, `CatalogResult.enough` ~L69-80. `apps/health/models.py`, `HealthProfile.age`, `clearance_is_valid`, `meal_schedule` ~L233-294 y `ConsentTextVersion.has_placeholders` ~L72.	Contraejemplo: `can_approve` y `simulated_blocked` se calculan en la vista (`apps/plans/views.py` ~L301-303) repitiendo la regla de `services.approve_plan` ~L499.	modelos; con excepciones en las vistas
G2	Creator	GRASP	Que cree el objeto quien lo contiene, lo usa o tiene los datos para inicializarlo.	`apps/plans/services.create_exercise_draft` ~L419-452; `apps/plans/nutrition.create_nutrition_draft` ~L88-123 y `_save_option` ~L71-83; `services.request_plan` ~L311-321.	La función que recibe la salida validada crea `Routine`, `RoutineDay`, `RoutineExercise`, `Plan`, `NutritionPlan`, `MealSlot`, `MealOption` y `PlanEvent`. El creador es una función de servicio, no el agregado contenedor (por ejemplo `NutritionPlan` no crea sus propios `MealSlot`).	Parcial
G3	Controller	GRASP	Un objeto, no de interfaz, recibe y coordina el evento del sistema.	Vistas delgadas: `apps/plans/views.trainer_plan_approve` ~L339-356 (valida el formulario, llama `services.approve_plan`, traduce `PlanError` a mensajes) y `member_meal_choice` ~L500-513. Controladores de casos de uso: `services.request_plan` ~L267, `approve_plan` ~L494, `nutrition.choose_option` ~L368, `health.services.save_profile` ~L275.	Las excepciones de dominio (`PlanError`, `ProfileError`, `ConsentError`) se traducen a mensajes en la vista, y la lógica queda en `services`. Las vistas de Django son el controlador del framework; la contribución del proyecto es la disciplina de mantenerlas delgadas. Hay vistas gruesas: `trainer_plan_inbox` ~L221-253, `_plan_detail_context` ~L277-312 y `apps/health/views.member_health_profile` ~L205-258.	Claro en la mayoría; parcial en tres vistas
G4	Low Coupling	GRASP	Mantener pocas y débiles dependencias entre elementos.	A favor: `apps/plans/rules/*` importan solo biblioteca estándar (por ejemplo `flags.py` ~L21-24); la IA queda tras `AIClient`. En contra: dependencia mutua `health` y `plans` (`health/models.py` ~L19, `health/services.py` ~L31 y `health/views.py` ~L26 importan `apps.plans.rules.flags`; `plans/services.py` ~L30-33 y `plans/jobs.py` ~L30	`nutrition.py` usa funciones privadas de `services` (`_check_trainer`, `_parse_token`, `_since`, `_archive`; ~L122, ~L224, ~L313, ~L422) y `views.py` ~L243 usa `services._since`. Los imports locales de esos tres puntos son síntoma de ciclos de diseño.	Parcial

Nº	Patrón	Familia	Intención	Dónde	Evidencia	Certeza
				importan `apps.health`); ciclo `services`, `jobs`, `nutrition` resuelto con imports locales (`services.py` ~L288 y ~L506, ambos con el comentario "evita el import circular", y `nutrition.py` ~L433).		
G5	High Cohesion	GRASP	Que cada módulo tenga responsabilidades enfocadas.	A favor: cada archivo de `rules/` tiene una sola preocupación (banderas, elegibilidad, catálogo, alimentos, energía, validación, adaptación); `ai/` se divide en `base`, `gemini`, `fake`, `prompts`; `notifications.py`. En contra: `services.py` (559 líneas: cupos, evaluación, pedidos, lectura, aprobación, borradores), `nutrition.py` (520: borrador, revisión, elección de la clienta, alternativas, compras), `jobs.py` (479: cola, reintentos, payloads y tres ejecutores), `views.py` (670).	La separación en `rules/` y `ai/` es muy nítida; los módulos de casos de uso agrupan varios temas.	Parcial
G6	Polymorphism	GRASP	Variar el comportamiento por tipo con operaciones polimórficas, sin condicionales.	`AIClient`, `FakeAIClient`, `GeminiClient` (`apps/plans/ai/`); `_RUNNERS` en `jobs.py` ~L475-479.	`jobs.py` ~L333 y ~L437 trata a cualquier cliente igual. Contraejemplo: `_call_with_retries` ~L236-246 discrimina por tipo de excepción con `except` en lugar de polimorfismo.	Claro (en `AIClient`)
G7	Pure Fabrication	GRASP	Crear una clase artificial, que no está en el dominio, para lograr cohesión y bajo acoplamiento.	`apps/plans/jobs.py` (cola, hilos, latidos, presupuesto de tiempo); `apps/plans/notifications.py`; módulos de servicios; `rules/*` como motor de reglas; `_SlotView` ~L381-385 (unifica la forma de una comida del plan nuevo y de "otra opción"); `apps/health/dates.py`.	Ninguno de esos elementos es un concepto del negocio del gimnasio; existen para repartir responsabilidades y facilitar las pruebas.	Claro
G8	Indirection	GRASP	Poner un intermediario entre dos elementos para desacoplarlos.	`get_ai_client()` entre `jobs` y los proveedores; `apps/health/dates.py` `today()` ~L9-10 entre el código y el reloj (su docstring dice que existe para simular el día en las pruebas); `adaptation.for_job` ~L189-210 entre `jobs`	Cada intermediario permite cambiar o simular lo que hay detrás sin tocar a quien lo llama.	Claro

Nº	Patrón	Familia	Intención	Dónde	Evidencia	Certeza
				y `rules/adaptation`; `jobs._sleep = time.sleep` ~L50 y `GeminiClient(opener=...)` ~L45 y ~L54 como puntos de sustitución.		
G9	Protected Variations	GRASP	Proteger el sistema de los puntos de variación o inestabilidad con una interfaz estable.	(a) Proveedor de IA: `AIClient` y normalización de errores; `GEMINI_SCHEMA_FIELD` configurable porque el proveedor cambia nombres de campos (`ai/gemini.py` ~L14-16, ~L29, ~L59-71). (b) Salida impredecible de la IA: `rules/validate.py` y `rules/meal_validate.py` validan todo; la IA solo elige ids y gramos y Python calcula energía y macros (`rules/foods.portion_totals` ~L171). (c) Reglas legales o profesionales cambiantes: `services.professional_required()` ~L400-403 y `health.services.ai_texts_ready()` ~L177-187 son interruptores únicos. (d) Cambios del modelo de datos: `ExerciseItem` y `FoodItem` aíslan a `rules/`.	En cada caso hay una frontera estable (interfaz, validador, interruptor, dataclass) delante de lo que puede cambiar.	Claro

1.3 Patrones GoF buscados y descartados (lo más cercano que hay)

Patrón	Lo más cercano	Por qué no cuenta
Singleton	`COLD_START` (`rules/adaptation.py` ~L151); `django.conf.settings`.	Es una constante inmutable compartida; nada controla que exista una sola instancia.
Abstract Factory	`get_ai_client()`.	Crea un solo producto, no familias de productos relacionados.
Builder	`build_exercise_payload` ~L181-226 y `build_meal_payload` ~L341-378 (`jobs.py`).	Arman un `dict` de una vez; no hay pasos separados ni director.
Prototype	`dataclasses.replace(eligibility, ...)` en `services.assess` ~L221, ~L231, ~L233.	Es copia con modificación de un valor inmutable, idioma de dataclasses.
Composite	`Plan`, `NutritionPlan`, `MealSlot`, `MealOption`, `MealOptionItem`; `Totals.__add__` ~L162.	Jerarquía de datos sin interfaz común de componente.
Bridge	`AIClient` frente a `jobs` y `prompts`.	Es Strategy; no hay dos jerarquías paralelas (abstracción e implementación).

Patrón	Lo más cercano	Por qué no cuenta
Proxy	`FakeAIClient`.	Es un doble de pruebas, no un sustituto que delegue con control de acceso o caché.
Flyweight	Regex precompiladas `_ALLERGEN_RE` y `_DIET_RE` (`meal_validate.py` ~L99-100).	Es caché a nivel de módulo, no objetos compartidos por el patrón.
Mediator	`jobs.py` como orquestador.	Los participantes no se comunican a través de él; es un orquestador de casos de uso (Pure Fabrication).
Chain of Responsibility (propia)	`flags.evaluate` ~L272-390 y `meal_validate.validate_meal_slot` ~L190-233.	Recorren todas las reglas acumulando errores; ningún manejador decide pasar a un sucesor.
Iterator, Visitor, Interpreter	`_compile(words)` (`meal_validate.py` ~L88-96).	Construye regex desde listas de palabras; no interpreta una gramática.

1.4 Otros patrones e idiomas relevantes (ni GoF ni GRASP)

Patrón o idioma	Dónde	Comentario
Service Layer	`apps/plans/services.py`, `nutrition.py`, `apps/health/services.py`	Los casos de uso viven aparte de las vistas y del ORM.
Núcleo funcional con cáscara imperativa	`apps/plans/rules/` (puro) frente a `services`, `jobs`, `views`	Las reglas no usan Django, base de datos ni red.
Objeto de valor inmutable	`Flag`, `Facts`, `Eligibility`, `Totals`, `AIResult` (dataclasses `frozen=True`)	Comparables y seguros de compartir.
Reglas guiadas por tablas	`_MISSING_TO_REQUIREMENT` (`eligibility.py` ~L46), `_FLAG_EXCLUDES`, `_CONDITION_EXCLUDES` (`catalog.py` ~L23-31), `_FLAG_TO_TAG` (`foods.py` ~L41), `MEAL_SPLITS` (`energy.py` ~L62)	Cambiar una regla es cambiar datos.
Receptor idempotente	`request_token` único (`models.py` ~L171) y `services.request_plan` ~L276-281	Recargar o hacer doble clic no crea otro trabajo.
Reclamo atómico (compare-and-set)	`jobs.claim` ~L81-90; restricción `plans_job_one_active` (`models.py` ~L209); `select_for_update` ~L328	Evita que dos procesos ejecuten el mismo trabajo.
Reintento con espera creciente	`jobs._call_with_retries` ~L229-247	Espera de 2, 6 y 18 s con variación y respeto de `Retry-After`.
Interruptores de función (Feature Toggle)	`health_features_enabled` (`health/services.py` ~L57), `ai_mode` (`ai/__init__.py` ~L23), `adaptation.enabled` (~L39), `professional_required` (`services.py` ~L400)	Funciones encendibles por configuración.
Bitácora de auditoría	`PlanEvent` (`models.py` ~L224-261), `HealthAccessLog` (`health/models.py` ~L297-325)	Registro de quién hizo qué, sin contenido de salud.

---

2. Patrones que pudo o debió aplicar y no aplicó

Nº	Patrón	Problema concreto (ejemplo del código)	Dónde se aplicaría	Qué mejoraría	Costo o riesgo	¿Vale la pena?
M1	State, al menos	Las reglas de transición están repartidas:	`Plan.status` y `PlanGeneratio`	Ampliar: agregar un estado (por	Bajo para la tabla; medio para clases	Tabla de transiciones:

Nº	Patrón	Problema concreto (ejemplo del código)	Dónde se aplicaría	Qué mejoraría	Costo o riesgo	¿Vale la pena?
	como tabla de transiciones	`take_for_review` solo desde `draft_ai` (`services.py` ~L465), `archive_plan` acepta pendientes y aprobados (~L487), `approve_plan` revisa dos veces (~L497 y ~L520), `regenerate_plan` (~L552), `edit_option` (`nutrition.py` ~L225), y `_archive` (~L474) y `approve_plan` (~L525) escriben el estado a mano. Hay 10 llamadas manuales a `PlanEvent.objects.create` (6 en `services.py`, 4 en `nutrition.py`). `valid_until` se guarda al aprobar (~L537) y solo se muestra (`views.py` ~L127-128): no hay estado ni transición por vencimiento.	`nJob.status`: un `ALLOWED_TRANSITIONS` y una función `transition(plan, nuevo_estado, actor, nota)` que valide y registre el evento.	ejemplo "vencido") hoy obliga a revisar unos 8 sitios. Probar: la tabla se prueba de una vez. Mantener: auditoría uniforme.	`State` completas. Riesgo: tocar el flujo de aprobación, que es sensible por razones legales; exige pruebas.	sí. Clases State completas: no, porque el comportamiento propio de cada estado es mínimo.
M2	Template Method (o una función común de cierre) para los tres ejecutores	`_run_exercise` ~L305-336, `_run_nutrition` ~L405-440 y `_run_alternative` ~L443-472 repiten: `_evaluate_for_run`, `_adaptation_for`, `update(flags=..., progress_total=...)`, `ai.get_ai_client()`, generar, `_check_budget` y el bloque `with transaction.atomic(): select_for_update; si status != RUNNING lanzar 'lost_lock'; crear; _finish` (idéntico en ~L327-335, ~L431-439 y ~L464-471).	`apps/plans/jobs.py`: helper `_commit(job, persist)` o clase base `PlanRunner` con `prepare`, `generate` y `persist`.	Mantener: un arreglo del bloqueo en un solo lugar. Ampliar: un cuarto tipo de trabajo. Probar: el esqueleto una vez.	Bajo con helper; medio con jerarquía. Riesgo: es código de concurrencia bajo `select_for_update`.	Sí para el helper; quizá para la jerarquía de clases.
M3	Template Method o Factory Method compartido para crear trabajos	`services.request_plan` ~L267-330 y `nutrition.request_alternative` ~L418-469 son dos copias del mismo flujo (token, idempotencia, trabajo activo, `recover_stale_jobs`, `assess`, rama de bloqueo, `create` dentro de `atomic`, `except IntegrityError`, `on_commit(start_job)`). Ya divergieron: `request_plan` evita repetir el aviso por el mismo bloqueo en el día (~L303-310) y `request_alternative` no (~L450-455), así que, según lo leído, cada clic en un caso bloqueado crearía otro trabajo y otro correo (no lo ejecuté).	`services.py`: `_create_job(client, kind, actor, token, extras, ...)` con ganchos para cupo y evento.	Mantener: corregir la carrera y la deduplicación una sola vez. Probar.	Bajo a medio; riesgo en la lógica de concurrencia.	Sí.

Nº	Patrón	Problema concreto (ejemplo del código)	Dónde se aplicaría	Qué mejoraría	Costo o riesgo	¿Vale la pena?
M4	Null Object (Special Case) para la adaptación	`adaptation` puede ser `None` y 10 líneas de `jobs.py` lo preguntan (~L190, ~L224, ~L298, ~L309, ~L311, ~L347, ~L376, ~L392, ~L394, ~L437). `adaptation.for_job` (~L189-210) devuelve `None` por cuatro motivos distintos (apagada, sin consentimiento, sin ficha, error).	`adaptation.for_job` devolvería un `JobAdaptation` vacío (`history=None`, `exercises=None`, `foods=None`, `summary()` vacío).	Mantener y probar: desaparecen unas 10 ramas.	Muy bajo; cuidar que el resumen vacío no se guarde (la guarda de ~L298 lo evita hoy).	Sí.
M5	Strategy o registro por tipo de plan (`PlanKind`)	Hay unas 24 comparaciones por tipo en `adaptation.py` (~L136-277, con 9 usos de `is_exercise`), `services.py` (~L217, ~L224, ~L505, ~L512), `nutrition.py` (~L165, ~L225, ~L262), `jobs.py` ~L163 y unas 5 más en `views.py` (~L284-302). La "alternativa" es un tercer tipo tratado a mano (`adaptation.for_job` usa `elif job.kind != KIND_EXERCISE`).	Un objeto por tipo con `assess_catalog`, `build_payload`, `schema`, `validate`, `create_draft`, `approval_problems` y `adaptation_slice`, registrado en un diccionario como `_RUNNERS`.	Ampliar: un tipo nuevo sería una clase más un registro, no 30 ediciones. Probar por tipo.	Medio a alto: toca los módulos centrales con riesgo de regresión en seguridad y reglas legales.	Quizá: hoy hay dos tipos; vale la pena si se planea un tercero (por ejemplo sueño o recuperación). Si no, bastan M2, M3 y M4.
M6	Specification única para "¿se puede aprobar?"	La regla se expresa en tres sitios: `views._plan_detail_context` ~L301-303 (`can_approve`, `simulated_blocked`), `services.approve_plan` ~L494-514 y `nutrition.approval_problems` ~L183-201. Si cambia una, la interfaz puede mostrar un botón que el servicio rechaza, o al revés.	`services.approval_blockers(plan, actor)` devuelve la lista de motivos; el servicio lanza con ella y la vista la pinta.	Mantener y probar: una sola fuente de la regla.	Bajo.	Sí.
M7	Facade o consulta única, con resultado calculado una vez por petición, para "alimentos y ejercicios permitidos hoy"	La composición `filter_foods(food_items(), nutrition_profile(...))` aparece en `assess` ~L225, `nutrition.current_allowed_ids` ~L158 y `exchange_rows` ~L491 (esta última con `plan.flags` guardadas, no las vigentes). En la pantalla de revisión de un plan de alimentación pendiente, `current_allowed_ids` corre hasta 3 veces (`review_context` ~L212, `approval_problems` ~L193 y ~L197), cada una leyendo ficha, banderas y toda la tabla `Food`. Algo parecido pasa con ejercicios en `routine_conflicts` ~L378-393.	`services.allowed_foods_for(client, profile=None)` y un objeto de contexto de revisión calculado una vez y pasado a `review_context` y `approval_problems`.	Mantener: un único criterio de "permitido". Rendimiento (por lectura del código, no medido). Probar.	Bajo; cuidar la diferencia de `exchange_rows` (usa `plan.flags`).	Sí.
M8	Decorator (objeto) o Proxy sobre `AIClient`	`_call_with_retries` ~L229-247 mezcla la política de reintentos con `_check_budget(started)` y `_heartbeat(job)`; el conteo	`ai/retry.py` con `RetryingClient(AIClient)` y ganchos	Probar los reintentos sin base de datos ni hilos. Ampliar con otro proveedor sin	Bajo; riesgo bajo a medio por el orden de los latidos.	Quizá: solo si se agrega un segundo proveedor o se reutiliza

Nº	Patrón	Problema concreto (ejemplo del código)	Dónde se aplicaría	Qué mejoraría	Costo o riesgo	¿Vale la pena?
	para reintentos y medición	de tokens vive en `_generate_validated` ~L264-265. `check_gemini` llama al cliente sin esos reintentos y copia a mano la forma del payload (`sample_payload` ~L43-59).	`before_call`; opcionalmente `MeteredClient`.	reescribir la política.		fuera de `jobs`; hoy hay un solo punto de llamada.
M9	Rule objects o Specification para las banderas de seguridad	`flags.evaluate` ~L272-390 es una función de unas 120 líneas con 44 condiciones, y los textos están aparte en `_TEXTS`. Los umbrales están marcados "[A VALIDAR]" en el propio archivo y cambiarán cuando los revise un profesional.	`rules/flags.py`: lista `RULES = [Rule(code, severidad, alcances, cuando=...)]` y un contexto que precalcule `bmi_value`, `clearance_valid` y `controlled_ok`.	Ampliar y probar regla por regla; revisión clínica regla por regla.	Medio. Riesgo alto: es código de seguridad de salud y exige pruebas de equivalencia exhaustivas (no vi las pruebas existentes).	Quizá: solo cuando el profesional pida cambiar muchas reglas. Mientras tanto, mantener la función con pruebas de tabla.
M10	Observer o eventos de dominio propios para avisos y auditoría	`jobs.run_job` ~L150-166 y `services`/`nutrition` llaman directamente a `notifications.*` y crean `PlanEvent` a mano; un canal nuevo (WhatsApp, push) obliga a tocar cada sitio. Los ciclos `services`, `jobs`, `nutrition` se evitan con imports locales (`services.py` ~L288, ~L506; `nutrition.py` ~L433).	Eventos `plan_approved`, `job_blocked`, `draft_ready`, con suscriptores `notifications` y `audit`.	Ampliar canales y romper ciclos de dependencia.	Medio; el flujo queda menos visible en un área con requisitos legales (qué se envía a quién).	Quizá, más bien no por ahora: hay cinco avisos y un solo canal. Sí si se agregan canales.
M11	Inyección de dependencias del cliente de IA (en lugar del localizador `get_ai_client()`)	`_run_exercise` ~L314, `_run_nutrition` ~L411 y `_run_alternative` ~L459 llaman a `ai.get_ai_client()`; para probar hay que parchear `apps.plans.ai.get_ai_client` (así lo indica el docstring de `ai/__init__.py`).	`run_job(job_id, client=None)` y `start_job(..., client=None)`.	Probar sin parches.	Muy bajo; cuidar el paso por hilos.	Quizá: `mock.patch` ya funciona, el beneficio es pequeño.
M12	Builder o DTO tipado para el payload de la IA	El contrato de "lo único que viaja a la IA" es un `dict` armado a mano en `build_exercise_payload` ~L181-226 y `build_meal_payload` ~L341-378. Las mismas claves se repiten como cadenas en `ai/fake.py` ~L25-50 y ~L79-110, en el texto de `ai/prompts.py` y en `check_gemini.sample_payload` ~L43-76 (copia manual).	Dataclasses `ExercisePayload` y `MealPayload` con `to_dict()` y lista blanca de campos.	Probar (una prueba de "sin campos personales" sobre la clase) y mantener (el fake y `check_gemini` reutilizan la misma estructura).	Bajo a medio.	Quizá: es frontera de privacidad, pero no sustituye las pruebas.
M13	Repositorio o QuerySet personalizado	Consultas como funciones sueltas (`_counted_jobs` ~L88, `active_job` ~L262, `approved_plan` ~L344, `pending_plans` ~L349, `plans_for_trainer` ~L335) y 13 consultas `PlanGenerationJob.objec`	`PlanGenerationJobQuerySet` (`active`, `counted`, `claim`) y `PlanQuerySet` (`approved`).	Mantener y probar.	Bajo; Django ya ofrece Manager y QuerySet.	Quizá: un QuerySet propio sí; una capa Repository formal, no.

Nº	Patrón	Problema concreto (ejemplo del código)	Dónde se aplicaría	Qué mejoraría	Costo o riesgo	¿Vale la pena?
		`ts.filter(pk=...)` en `jobs.py`, casi todas con `update` de estado.				
M14	Pipeline o Chain of Responsibility para validar la salida de la IA	`validate_exercise_plan` (~75 líneas) y `validate_meal_slot` (~45) mezclan esquema, textos, ids, rangos y variedad.	`rules/validate.py` y `rules/meal_validate.py`.	Poco: ya reutilizan `check_text`, `_exact_keys` y `_option_rows`.	Añadiría indirección sin variación real; el orden de las comprobaciones importa.	No.
M15	Strategy para los cupos de uso	`global_quota_ok`, `client_quota_ok`, `trainer_quota_ok` (`services.py` ~L93-117) y `alternatives_left` (`nutrition.py` ~L309-315) repiten el patrón "contar trabajos en una ventana de tiempo".	`services.py`.	Un helper `_count_jobs(window, **filtros)` basta.	Muy bajo.	No como patrón; sí como pequeña extracción de método.

---

3. Resumen

- Predominan los patrones de asignación de responsabilidades (GRASP) y de arquitectura (capa de servicios, núcleo de reglas puro en `rules/`, objetos de valor inmutables, reglas guiadas por tablas) sobre los patrones GoF de manual.
- De GoF aparecen pocos y con claridad: Strategy/Polymorphism en `AIClient` (`FakeAIClient` y `GeminiClient`), Adapter en `GeminiClient` y en el mapeo del ORM a dataclasses, Facade parcial en `assess` y Template Method funcional en `_generate_validated`. Command, Observer y los Template Method de formularios y admin existen sobre todo porque Django los aporta.
- Los creacionales casi no existen (solo una fábrica simple, `get_ai_client`); Singleton, Builder y Abstract Factory no aparecen y no hacen falta.
- Lo que más falta: un ciclo de vida explícito para `Plan` y `PlanGenerationJob` (hoy son condiciones dispersas), una abstracción por tipo de plan (unas 30 comparaciones por `kind` y tres ejecutores casi iguales) y un Null Object para la adaptación (10 comprobaciones de `None`).
- Hay lógica duplicada que ya empezó a divergir (`request_plan` frente a `request_alternative`; la regla de aprobación en vista y servicio; la composición del filtro de alimentos en tres sitios), lo que pesa más que cualquier patrón ausente.
- El acoplamiento es la parte más débil: dependencia mutua `health` y `plans`, ciclos entre `services`, `jobs` y `nutrition` resueltos con imports locales, y uso de funciones privadas entre módulos.
- Prioridad sugerida: M3, M2, M4, M6 y M7 (baratos y de bajo riesgo), luego M1 (tabla de transiciones). Dejar para cuando haya un motivo concreto M5, M9 y M10, y no

reescribir `flags.evaluate` sin pruebas de equivalencia.

- Limitación: análisis estático sin ejecutar el sistema ni revisar las pruebas; las afirmaciones sobre rendimiento y sobre la divergencia de M3 salen de leer el código.

Anexo C. Prompts del experimento

C.1 Prompt A (control), íntegro

Reconstrucción del prompt original. Archivo `anexos/C-prompt-control-A.md`.

```
Eres un desarrollador de software. Trabajas en un proyecto Django existente (ProFit
Studio, gestión de un gimnasio) ubicado en <RAIZ_DEL_PROYECTO> (usa
`.venv/Scripts/python.exe`; estás en Windows con Git Bash).

## Qué construir

Los planes de ejercicio o alimentación de una clienta (`apps/plans/models.py`: `Plan`,
`PlanEvent`) los genera una IA como borrador y los revisa un entrenador. Implementa el
módulo del CICLO DE VIDA DEL PLAN (los cambios de estado de un `Plan`) en
`docs/actividad-patrones/anexos/experimento/exp_a/plan_lifecycle.py`, con sus pruebas
en `docs/actividad-patrones/anexos/experimento/exp_a/tests.py` (crea también
`exp_a/__init__.py`).

Estados de `Plan.status`: `draft_ai` (borrador de la IA), `in_review` (en revisión),
`approved` (aprobado: la clienta lo ve), `superseded` (reemplazado por uno aprobado
más nuevo) y `archived` (archivado).

Operaciones públicas (firmas fijas; el resto del sistema las usará así):

- `take_for_review(plan, actor) -> bool`: `draft_ai` -> `in_review`. Guarda
`reviewed_by=actor` y registra un `PlanEvent` con acción `PlanEvent.ACTION_TAKEN`.
Devuelve `True` si cambió el estado; si el plan no estaba en `draft_ai` no hace nada y
devuelve `False`.
- `archive_plan(plan, actor, reason='') -> Plan`: desde `draft_ai`, `in_review` o
`approved` pasa a `archived`. Guarda `archived_at`, desactiva su rutina
(`Routine.is_active=False`) si tiene y registra un evento `ACTION_ARCHIVED` con el
motivo (se recorta a 300 caracteres; si viene vacío, ``'Archivado por el
entrenador.'``). Desde cualquier otro estado lanza `PlanStateError('Este plan ya no se
puede archivar.')`.
- `approve_plan(plan, actor, *, note_for_client='', warnings_ack=False,
stale_ack=False) -> Plan`: `draft_ai` o `in_review` -> `approved`. Reglas previas: si
el plan no está pendiente, `PlanStateError('Solo se puede aprobar un borrador o un
plan en revisión.')`; si `plan.is_simulated` y no `settings.DEBUG`,
`PlanStateError('Este plan se generó en MODO DE PRUEBA: no se puede aprobar. Genera
uno real.')`; si `plan.flags` no está vacío y no `warnings_ack`,
`PlanStateError('Marca "Revisé las advertencias" para aprobar.')`. (`stale_ack` se
acepta pero este módulo no lo usa.) Efectos, todos dentro de una transacción con
`select_for_update` y volviendo a verificar el estado dentro de ella (si cambió:
`PlanStateError('Este plan cambió mientras lo revisabas. Recarga la página.')`): el
plan aprobado anterior de la misma clienta y del mismo tipo pasa a `superseded` (se
desactiva su rutina y se registra un evento `ACTION_SUPERSEDED` con la nota
`f'Reemplazado por la versión {plan.version}.'``); el plan queda `approved` con
`approved_by=actor`, `approved_at=ahora`, `reviewed_by` (el que ya tenía o el actor),
`note_for_client` (recortada a 500), `valid_until = hoy + 28 días` (usa
`apps.health.dates.today()`), y si tenía `flags` guarda `warnings_ack_by` y
`warnings_ack_at`; su rutina se activa (`is_active=True`); se registra un evento
`ACTION_APPROVED`; al confirmarse la transacción se avisa a la clienta con
`apps.plans.notifications.client_plan_approved(plan)` (usa `transaction.on_commit`).
- `regenerate_plan(plan, actor, notes, request_token) -> (job, created)`: solo desde
`draft_ai` o `in_review` (si no, `PlanStateError('Solo se regenera un borrador o un
plan en revisión.')`). Pide un trabajo de generación con
`apps.plans.services.request_plan(plan.client, plan.kind, actor, request_token,
```

```
trainer_notes=notes, parent_plan=plan)` (úsala como caja negra: ya existe) y, si
`created` es verdadero, registra un evento `ACTION_REGENERATE` con las notas
(recortadas a 500).

Todas las operaciones verifican primero que el `actor` sea entrenador con acceso a la
clienta del plan (`actor.is_trainer` y que la clienta esté en
`apps.accounts.permissions.clients_for_trainer(actor)`); si no, `PlanNotAllowed('No
tienes acceso a este plan.')`.

Quedan fuera de este módulo (las comprueba quien llama): la credencial de nutrición,
que la rutina tenga ejercicios y la ficha desactualizada.

Reutiliza lo que ya existe: `Plan` y `PlanEvent` (`apps/plans/models.py`), `Routine`
(`apps/routines/models.py`) y las excepciones `PlanNotAllowed` y `PlanStateError` de
`apps/plans/services.py`.

## Pautas de diseño

- Clean Architecture: separa responsabilidades (controladores HTTP, servicios de
negocio, acceso a datos).
- Mensajes de error genéricos y un `logger` para registrar los cambios de estado.
- Código mantenible y con pruebas.

## Reglas de esta tarea

- No modifiques ningún archivo existente del proyecto. Solo crea los 3 archivos de
`exp_a/` indicados.
- No leas las funciones de ciclo de vida que ya existen en `apps/plans/services.py`
(`take_for_review`, `_archive`, `archive_plan`, `approve_plan`, `regenerate_plan`).
Puedes ver ese archivo únicamente con `grep -n "^class "` para conocer las excepciones
y, si lo necesitas, las constantes del principio del archivo. Tampoco leas
`apps/plans/nutrition.py`, `apps/plans/jobs.py` ni nada dentro de `docs/` (salvo este
archivo). Sí puedes leer los modelos, `apps/accounts/permissions.py` y los helpers de
pruebas de `apps/plans/tests/helpers.py` (por ejemplo `PlanScenarioTestCase`).
- Las pruebas se corren desde la raíz del proyecto con: `PYTHONPATH=docs/actividad-
patrones/anexos/experimento .venv/Scripts/python.exe manage.py test exp_a`. No uses `-
-parallel`. No ejecutes la suite completa del proyecto.
- No hagas commits, ni push, ni cambies de rama (git solo lectura). No instales
dependencias.

## Respuesta final (breve)

1. La primera línea: el ID exacto del modelo que eres.
2. Archivos creados, número de líneas de `plan_lifecycle.py`, número de pruebas y
resultado (OK o fallos).
3. Lista exacta de los archivos del proyecto que leíste.
4. Decisiones de diseño que tomaste (máximo 8 líneas).
```

C.2 Prompt B (reformulado): única diferencia con A

B es igual a A salvo el nombre de la carpeta de salida (`exp_b` en lugar de `exp_a`) y este párrafo adicional en «Pautas de diseño» (verificado con `diff`). Archivo `anexos/D-prompt-reformulado-B.md`.

```
- Aplica el patrón de diseño State (GoF) para modelar el ciclo de vida del plan: cada
estado (`draft_ai`, `in_review`, `approved`, `superseded`, `archived`) es una clase
que conoce qué operaciones admite y a qué estado transiciona, y el plan (el contexto)
delega en su estado actual. Evita los condicionales dispersos por estado (`if
```

```
plan.status in ...`) dentro de las operaciones públicas. Conserva exactamente las
firmas públicas, las reglas y los mensajes indicados arriba.
```

C.3 Prompt de ampliación (estado «vencido»), versión para A

La versión para B es idéntica salvo la carpeta (`exp_b`). Archivos `anexos/E-prompt-ampliacion-A.md` y `E-prompt-ampliacion-B.md`.

```
Eres un desarrollador de software. Trabajas en un proyecto Django existente (ProFit
Studio, gestión de un gimnasio) ubicado en <RAIZ_DEL_PROYECTO> (usa
`.venv/Scripts/python.exe`; estás en Windows con Git Bash).

## Situación

Ya existe el módulo del ciclo de vida de un `Plan` (los cambios de estado de un plan
de ejercicio o alimentación) en `docs/actividad-patrones/anexos/experimento/exp_a/`:
`plan_lifecycle.py` y `tests.py`. Léelos completos antes de empezar.

## Cambio pedido: nuevo estado «vencido» (`expired`)

- Un plan `approved` cuya fecha `valid_until` es anterior a hoy debe poder pasar a
`expired`. Agrega la operación pública `expire_overdue_plans(today=None) -> int`:
marca como `expired` todos los planes `approved` con `valid_until < today` (por
defecto `apps.health.dates.today()`), desactiva su rutina (`Routine.is_active=False`),
registra para cada uno un `PlanEvent` con la acción `'expired'` (es una cadena nueva;
cabe en el campo, no cambies el modelo) y la nota `'Vigencia vencida.'`, y devuelve
cuántos cambió. Es idempotente: no toca los planes que ya no están en `approved`.
Registra el cambio con el `logger`.
- Un plan `expired` se puede archivar con `archive_plan` (pasa a `archived`, mismo
resultado que archivar un aprobado; su rutina ya está inactiva). Un plan `expired` NO
se puede aprobar, ni tomar para revisión, ni regenerar: cada operación responde como
ya responde para un estado no permitido (los mismos mensajes y valores de retorno que
existen hoy).
- Un plan `expired` no cuenta como aprobado vigente: al aprobar otro plan de la misma
clienta y tipo, el `expired` no se reemplaza ni se modifica.
- Las operaciones y los mensajes que ya existen no cambian de comportamiento para los
demás estados.
- Actualiza `tests.py`: agrega las pruebas del estado nuevo y no rompas las
existentes.

## Reglas de esta tarea

- Edita únicamente los archivos de `exp_a/` (`plan_lifecycle.py` y `tests.py`). No
hagas más cambios de los necesarios para este pedido: no reorganices ni renombres lo
que no haga falta.
- No leas ninguna otra carpeta `exp_*`, ni nada dentro de `docs/` salvo este archivo y
`exp_a/`. Puedes leer los modelos (`apps/plans/models.py`), `apps/health/dates.py`,
`apps/routines/models.py` y `apps/plans/tests/helpers.py`.
- Las pruebas se corren desde la raíz del proyecto con: `PYTHONPATH=docs/actividad-
patrones/anexos/experimento .venv/Scripts/python.exe manage.py test exp_a`. No uses `-
-parallel` ni ejecutes la suite completa del proyecto.
- No hagas commits, ni push, ni cambies de rama (git solo lectura). No instales
dependencias.

## Respuesta final (breve)

1. La primera línea: el ID exacto del modelo que eres.
2. Lista de los lugares del código que tuviste que tocar (función, clase o constante)
y por qué.
```

```
3. Número total de pruebas y resultado (OK o fallos).
4. Cualquier dificultad o duda de diseño (máximo 5 líneas).
```

Anexo D. Fragmentos de código comparados

Los archivos completos están en `anexos/experimento/`: las versiones generadas en `base/exp_a` y `base/exp_b` y las ampliadas en `exp_a` y `exp_b`.

D.1 Original (V0): `apps/plans/services.py`, líneas 482 a 503

```
    plan.status = Plan.STATUS_ARCHIVED
    plan.archived_at = timezone.now()
    plan.save(update_fields=['status', 'archived_at'])
    if plan.routine_id:
        Routine.objects.filter(pk=plan.routine_id).update(is_active=False)
    PlanEvent.objects.create(plan=plan, actor=actor, action=PlanEvent.ACTION_ARCHIVED,
note=reason[:500])


def archive_plan(plan, actor, reason=''):
    """Descarta un borrador (o retira un plan aprobado: su rutina deja de verse)."""
    _check_trainer(plan, actor)
    with transaction.atomic():
        plan = Plan.objects.select_for_update().get(pk=plan.pk)
        if plan.status not in (*Plan.PENDING_STATUSES, Plan.STATUS_APPROVED):
            raise PlanStateError('Este plan ya no se puede archivar.')
        _archive(plan, actor, (reason or '').strip()[:ARCHIVE_REASON_MAX] or 'Archivado por
el entrenador.')
    logger.info('plan archived plan_id=%s actor_id=%s', plan.pk, actor.pk)
    return plan


def approve_plan(plan, actor, *, note_for_client='', warnings_ack=False, stale_ack=False):
    """Aprueba un borrador: la clienta lo ve desde ahora. Reemplaza al aprobado
anterior."""
```

D.2 Control A: constantes y `archive_plan` (`plan_lifecycle.py`, líneas 43-47 y 144-161)

```
ARCHIVABLE_STATUSES = (Plan.STATUS_DRAFT_AI, Plan.STATUS_IN_REVIEW, Plan.STATUS_APPROVED)
APPROVAL_VALIDITY_DAYS = 28
ARCHIVE_REASON_MAX = 300
NOTE_MAX = 500
DEFAULT_ARCHIVE_REASON = 'Archivado por el entrenador.'
…
def archive_plan(plan, actor, reason=''):
    """Archiva un borrador, un plan en revisión o un plan aprobado y desactiva su
rutina."""
    _ensure_access(plan, actor)
    with transaction.atomic():
        locked = _lock_plan(plan.pk)
        if locked.status not in ARCHIVABLE_STATUSES:
            raise PlanStateError(MSG_CANNOT_ARCHIVE)
        previous_status = locked.status
        locked.status = Plan.STATUS_ARCHIVED
        locked.archived_at = timezone.now()
        locked.save(update_fields=['status', 'archived_at'])
        _set_routine_active(locked, False)
        _record_event(
            locked, actor, PlanEvent.ACTION_ARCHIVED, _clip(reason, ARCHIVE_REASON_MAX) or
DEFAULT_ARCHIVE_REASON,
        )
    plan.refresh_from_db()
```

```
    logger.info('Plan %s: %s -> archived (actor %s).', plan.pk, previous_status, actor.pk)
    return plan
```

D.3 B con State: estados y contexto (`plan_lifecycle.py`, líneas 90-127, 186-194 y 262-274)

```
class PlanState:
    """Estado base: no admite ninguna operación. Los estados concretos sobrescriben solo
las suyas."""
    status = None
    entered_action = None     # evento que se registra al entrar en este estado
    routine_active = None     # None = no toca la rutina; True/False = la activa/desactiva
al entrar

    def take_for_review(self, ctx, actor):
        return False

    def archive(self, ctx, actor, reason):
        raise PlanStateError(MSG_CANNOT_ARCHIVE)

    def approve(self, ctx, actor, *, note_for_client, warnings_ack):
        raise PlanStateError(MSG_CANNOT_APPROVE)

    def ensure_approvable_under_lock(self):
        """Segunda verificación, ya con la fila bloqueada: solo un estado pendiente la
supera."""
        raise PlanStateError(MSG_CHANGED)

    def regenerate(self, ctx, actor, notes, request_token):
        raise PlanStateError(MSG_CANNOT_REGENERATE)

    def supersede(self, ctx, actor, replacement):
        raise PlanStateError(MSG_CHANGED)


class _ArchivableState(PlanState):
    """Estados desde los que se puede archivar: pasan a `archived`."""

    def archive(self, ctx, actor, reason):
        with ctx.locked():
            ctx.ensure_state(self, MSG_CANNOT_ARCHIVE)
            ctx.transition(
                Plan.STATUS_ARCHIVED, actor,
                note=_clip(reason, ARCHIVE_REASON_MAX) or DEFAULT_ARCHIVE_NOTE,
                archived_at=timezone.now(),
            )
        return ctx.plan
…
class ApprovedState(_ArchivableState):
    status = Plan.STATUS_APPROVED
    entered_action = PlanEvent.ACTION_APPROVED
    routine_active = True

    def supersede(self, ctx, actor, replacement):
        ctx.transition(
            Plan.STATUS_SUPERSEDED, actor, note=f'Reemplazado por la versión
{replacement.version}.')

…
    def transition(self, target_status, actor, *, note='', **fields):
        """Pasa el plan a `target_status`, guarda `fields`, ajusta la rutina y registra el
evento."""
        plan = self.plan
```

```
        origin = plan.status
        target = state_for(target_status)
        plan.status = target.status
        for name, value in fields.items():
            setattr(plan, name, value)
        self.repository.save(plan, ['status', *fields])
        if target.routine_active is not None:
            self.repository.set_routine_active(plan, target.routine_active)
        self.repository.add_event(plan, actor, target.entered_action, note)
        logger.info('Plan %s: %s -> %s (usuario %s).', plan.pk, origin, target.status,
actor.pk)
```

D.4 Diff de la ampliación en A (`plan_lifecycle.py`)

```
--- docs/actividad-patrones/anexos/experimento/base/exp_a/plan_lifecycle.py     2026-10-15
16:58:31.319097300 -0500
+++ docs/actividad-patrones/anexos/experimento/exp_a/plan_lifecycle.py  2026-10-15
16:59:12.918002100 -0500
@@ -1,6 +1,8 @@
 """Ciclo de vida de un `Plan`: los cambios de estado (capa de casos de uso).

-Estados: `draft_ai` -> `in_review` -> `approved` -> `superseded` / `archived`.
+Estados: `draft_ai` -> `in_review` -> `approved` -> `superseded` / `archived`. Un plan
`approved`
+cuya vigencia (`valid_until`) ya pasó se marca `expired` (`expire_overdue_plans`) y luego
solo se
+puede archivar.

 Capas de este módulo (de adentro hacia afuera):

@@ -8,7 +10,8 @@
 2. Acceso a datos: funciones pequeñas con prefijo `_` que leen o escriben `Plan`,
`PlanEvent` y
    `Routine`. Los casos de uso no escriben consultas sueltas.
 3. Autorización: `_ensure_access` (el actor debe ser entrenador con acceso a la clienta).
-4. Casos de uso públicos: `take_for_review`, `archive_plan`, `approve_plan`,
`regenerate_plan`.
+4. Casos de uso públicos: `take_for_review`, `archive_plan`, `approve_plan`,
`regenerate_plan`,
+   `expire_overdue_plans`.

 Las vistas HTTP (controladores) solo traducen la petición a estas llamadas y las
excepciones a
 respuestas. Aquí no hay `request` ni plantillas. Quedan fuera, porque las comprueba quien
llama:
@@ -33,18 +36,22 @@

 __all__ = [
     'PlanNotAllowed', 'PlanStateError',
-    'take_for_review', 'archive_plan', 'approve_plan', 'regenerate_plan',
+    'take_for_review', 'archive_plan', 'approve_plan', 'regenerate_plan',
'expire_overdue_plans',
 ]

 logger = logging.getLogger(__name__)

 # ── 1. Reglas puras ──────────────────────────────────────────────────────

-ARCHIVABLE_STATUSES = (Plan.STATUS_DRAFT_AI, Plan.STATUS_IN_REVIEW, Plan.STATUS_APPROVED)
+# El modelo `Plan` no define el estado `expired` (este módulo no lo toca): se declara
aquí.
+STATUS_EXPIRED = 'expired'
+ACTION_EXPIRED = 'expired'
```

```
+ARCHIVABLE_STATUSES = (Plan.STATUS_DRAFT_AI, Plan.STATUS_IN_REVIEW, Plan.STATUS_APPROVED,
STATUS_EXPIRED)
 APPROVAL_VALIDITY_DAYS = 28
 ARCHIVE_REASON_MAX = 300
 NOTE_MAX = 500
 DEFAULT_ARCHIVE_REASON = 'Archivado por el entrenador.'
+EXPIRED_NOTE = 'Vigencia vencida.'

 MSG_NOT_ALLOWED = 'No tienes acceso a este plan.'
 MSG_CANNOT_ARCHIVE = 'Este plan ya no se puede archivar.'
@@ -100,6 +107,13 @@
         logger.info('Plan %s: approved -> superseded por el plan %s.', old.pk,
new_plan.pk)


+def _lock_overdue_approved(today):
+    """Planes aprobados con vigencia anterior a `today`, con la fila bloqueada (sin fecha
= no vence)."""
+    return list(
+        Plan.objects.select_for_update().filter(status=Plan.STATUS_APPROVED,
valid_until__lt=today)
+    )
+
+
 def _notify_client_approved(plan):
     """Aviso a la clienta tras confirmarse la transacción. Un fallo del aviso no deshace
la aprobación."""
     try:
@@ -142,7 +156,7 @@


 def archive_plan(plan, actor, reason=''):
-    """Archiva un borrador, un plan en revisión o un plan aprobado y desactiva su
rutina."""
+    """Archiva un borrador, un plan en revisión, un plan aprobado o uno vencido y
desactiva su rutina."""
     _ensure_access(plan, actor)
     with transaction.atomic():
         locked = _lock_plan(plan.pk)
@@ -225,3 +239,23 @@
     if created:
         logger.info('Plan %s: regeneración pedida (actor %s).', plan.pk, actor.pk)
     return job, created
+
+
+def expire_overdue_plans(today=None):
+    """`approved` -> `expired` para los planes cuya `valid_until` es anterior a `today`.
Devuelve cuántos.
+
+    `today` es hoy (`dates.today()`) si no se indica. Desactiva la rutina de cada plan y
registra su
+    evento sin actor (lo hace el sistema, no una persona). Es idempotente: solo toca
planes que
+    siguen en `approved`, así que repetirlo no cambia nada.
+    """
+    if today is None:
+        today = dates.today()
+    with transaction.atomic():
+        overdue = _lock_overdue_approved(today)
+        for plan in overdue:
+            plan.status = STATUS_EXPIRED
+            plan.save(update_fields=['status'])
+            _set_routine_active(plan, False)
+            _record_event(plan, None, ACTION_EXPIRED, EXPIRED_NOTE)
```

```
+            logger.info('Plan %s: approved -> expired (vigencia hasta %s).', plan.pk,
plan.valid_until)
+    return len(overdue)
```

D.5 Diff de la ampliación en B (`plan_lifecycle.py`)

```
--- docs/actividad-patrones/anexos/experimento/base/exp_b/plan_lifecycle.py     2026-10-15
16:58:31.414924800 -0500
+++ docs/actividad-patrones/anexos/experimento/exp_b/plan_lifecycle.py  2026-10-15
16:59:31.552991100 -0500
@@ -3,11 +3,14 @@
 Capas (Clean Architecture; los controladores HTTP quedan fuera y solo llaman a las
funciones públicas):

 - Servicios de negocio (abajo del todo): `take_for_review`, `archive_plan`, `approve_plan`
y
-  `regenerate_plan`. Comprueban el acceso del entrenador y delegan en el contexto.
+  `regenerate_plan`, que comprueban el acceso del entrenador y delegan en el contexto, y
+  `expire_overdue_plans`, que corre sin entrenador (proceso programado) y marca como
vencidos
+  los planes aprobados cuya vigencia ya pasó.
 - Dominio: `PlanLifecycle` (el contexto) delega cada operación en la clase del estado
actual del
-  plan. Cada estado (`DraftAIState`, `InReviewState`, `ApprovedState`, `SupersededState`,
-  `ArchivedState`) declara qué operaciones admite y a qué estado pasa; lo que no admite lo
-  rechaza la clase base `PlanState`. No hay `if plan.status ...` en las operaciones
públicas.
+  plan. Cada estado (`DraftAIState`, `InReviewState`, `ApprovedState`, `ExpiredState`,
+  `SupersededState`, `ArchivedState`) declara qué operaciones admite y a qué estado pasa;
lo que
+  no admite lo rechaza la clase base `PlanState`. No hay `if plan.status ...` en las
operaciones
+  públicas.
 - Acceso a datos: `PlanRepository` (consultas, bloqueos de fila, rutina y eventos).

 Quedan fuera (las comprueba quien llama): la credencial de nutrición, que la rutina tenga
@@ -42,6 +45,11 @@
 MSG_CANNOT_REGENERATE = 'Solo se regenera un borrador o un plan en revisión.'
 MSG_UNKNOWN_STATE = 'Este plan está en un estado desconocido.'
 DEFAULT_ARCHIVE_NOTE = 'Archivado por el entrenador.'
+EXPIRED_NOTE = 'Vigencia vencida.'
+
+# El modelo no define el estado «vencido» ni su acción; ambas cadenas caben en sus campos.
+STATUS_EXPIRED = 'expired'
+ACTION_EXPIRED = 'expired'


 def _clip(text, limit):
@@ -71,6 +79,10 @@
             .exclude(pk=plan.pk)
         )

+    def overdue_approved(self, today):
+        """Planes aprobados cuya vigencia terminó antes de `today` (sin bloquear)."""
+        return list(Plan.objects.filter(status=Plan.STATUS_APPROVED,
valid_until__lt=today).order_by('pk'))
+
     def save(self, plan, fields):
         plan.save(update_fields=fields)

@@ -112,6 +124,10 @@
     def supersede(self, ctx, actor, replacement):
         raise PlanStateError(MSG_CHANGED)
```

```
+    def expire(self, ctx, actor):
+        """Solo un plan aprobado vence; en cualquier otro estado no hace nada
(idempotente)."""
+        return False
+

 class _ArchivableState(PlanState):
     """Estados desde los que se puede archivar: pasan a `archived`."""
@@ -192,6 +208,20 @@
         ctx.transition(
             Plan.STATUS_SUPERSEDED, actor, note=f'Reemplazado por la versión
{replacement.version}.')

+    def expire(self, ctx, actor):
+        with ctx.locked():
+            if ctx.state is not self:
+                return False
+            ctx.transition(STATUS_EXPIRED, actor, note=EXPIRED_NOTE)
+        return True
+
+
+class ExpiredState(_ArchivableState):
+    """Plan aprobado cuya vigencia terminó: solo se puede archivar (la rutina ya está
inactiva)."""
+    status = STATUS_EXPIRED
+    entered_action = ACTION_EXPIRED
+    routine_active = False
+

 class SupersededState(PlanState):
     status = Plan.STATUS_SUPERSEDED
@@ -207,7 +237,8 @@

 _STATES = {
     state.status: state
-    for state in (DraftAIState(), InReviewState(), ApprovedState(), SupersededState(),
ArchivedState())
+    for state in (
+        DraftAIState(), InReviewState(), ApprovedState(), ExpiredState(),
SupersededState(), ArchivedState())
 }


@@ -247,6 +278,9 @@
     def supersede(self, actor, replacement):
         return self.state.supersede(self, actor, replacement)

+    def expire(self, actor=None):
+        return self.state.expire(self, actor)
+
     # Apoyo para los estados
     @contextmanager
     def locked(self):
@@ -271,11 +305,11 @@
         if target.routine_active is not None:
             self.repository.set_routine_active(plan, target.routine_active)
         self.repository.add_event(plan, actor, target.entered_action, note)
-        logger.info('Plan %s: %s -> %s (usuario %s).', plan.pk, origin, target.status,
actor.pk)
+        logger.info('Plan %s: %s -> %s (usuario %s).', plan.pk, origin, target.status,
getattr(actor, 'pk', None))

     def record(self, actor, action, note=''):
         self.repository.add_event(self.plan, actor, action, note)
-        logger.info('Plan %s: evento %s (usuario %s).', self.plan.pk, action, actor.pk)
```

```
+        logger.info('Plan %s: evento %s (usuario %s).', self.plan.pk, action,
getattr(actor, 'pk', None))

     def supersede_previous(self, actor):
         """Reemplaza al plan aprobado anterior de la misma clienta y tipo (si lo hay)."""
@@ -306,7 +340,7 @@


 def archive_plan(plan, actor, reason=''):
-    """Archiva un borrador, un plan en revisión o uno aprobado."""
+    """Archiva un borrador, un plan en revisión, uno aprobado o uno vencido."""
     return _lifecycle_for(plan, actor).archive(actor, reason)


@@ -320,7 +354,21 @@
     return _lifecycle_for(plan, actor).regenerate(actor, notes, request_token)


+def expire_overdue_plans(today=None):
+    """Pasa a `expired` los planes `approved` con `valid_until` anterior a `today` (por
defecto, hoy).
+
+    Desactiva su rutina y registra el evento `expired` de cada uno (sin autor: lo hace el
sistema).
+    Devuelve cuántos cambió. Es idempotente: los planes que ya no están aprobados no se
tocan.
+    """
+    if today is None:
+        today = health_dates.today()
+    repository = PlanRepository()
+    expired = sum(1 for plan in repository.overdue_approved(today) if PlanLifecycle(plan,
repository).expire())
+    logger.info('Planes vencidos al %s: %s.', today, expired)
+    return expired
+
+
 __all__ = [
     'PlanLifecycle', 'PlanRepository', 'PlanState', 'state_for',
-    'take_for_review', 'archive_plan', 'approve_plan', 'regenerate_plan',
+    'take_for_review', 'archive_plan', 'approve_plan', 'regenerate_plan',
'expire_overdue_plans',
 ]
```

Anexo E. Registro de verificaciones

Tabla E1. Comprobaciones hechas por el equipo (las ejecutó Claude Code)

Qué se comprobó	Cómo	Resultado
Commits y firmas de modelo	`git rev-list --count`; `git log` con búsqueda de «Co-Authored-By»	78 commits; Sonnet 5.5: 31, Sonnet 4.6: 23, Opus 5.5: 1
Tamaño del código de producción	`wc -l` sin pruebas ni migraciones	`plans`: 6 344 líneas; `health`: 1 901; 46 archivos `.py`
Cifras de la consulta	`grep -c`, `git log -S`, lectura de las funciones citadas	Ver Tabla 3 del informe
Menciones de patrones en el repositorio	`git grep -c -iE` con los nombres de patrones	Sin resultados
Prompts A y B; ampliación en A y B	`diff` entre archivos	Solo cambian el párrafo de State y el nombre de la carpeta
Pruebas de A y B tal como se generaron	`PYTHONPATH=anexos/experimento manage.py test exp_a` (y `exp_b`)	A: 45 OK. B: 51 OK
Pruebas de A y B tras ampliar	Mismo comando	A: 67 OK (13,6 s). B: 73 OK (12,4 s)
Diff de la ampliación	`diff -u` contra la copia guardada antes de ampliar	A: +34 y −5 (módulo), +210 y −3 (pruebas). B: +47 y −9 (módulo), +178 y −1 (pruebas)
Que el experimento no alterara el proyecto	`git status` y suite completa `manage.py test`	Solo archivos nuevos en `docs/actividad-patrones/`. Suite completa del proyecto tras el experimento: 1009 pruebas OK (178 s), las mismas que antes.